

AZURE-DE (Databricks & PySpark)

7. Introduction to Databricks	10. Bigdata Fundamentals	11. Databricks & PySpark	12. DeltaLake & Lakehouse
13. Databricks	14. Spark Structured Streaming		

7.Introduction to Databricks(3):

1. [Introduction to databricks:](#)

Its platform to process big data by running spark jobs with much extra utilities

2. [Community Edition Signup:](#)

without select clouds, by click personal(community edition) in databricks sign up process.

3. [Databricks platform overview:](#)

compute(driver, worker Machines for processing spark jobs),workspace(Notebooks,Folders),users(emails),catalog(data organiser, schemas,tables,views),workflows,DLT...etc

10.Bigdata Fundamentals(6):

1. [Introduction to Bigdata:](#)

Volume, Velocity, Variety, Veracity(Authenticity, Accuracy/quality of data)

2. [Evolution of Bigdata:](#)

by vast growing of public needs, producing data evolved in exponential ways in massive manner upto Peta Bytes, Exa Bytes...etc and to process this, more tools/frameworks entered into market hadoop, spark, databricks along with Famous flexible Cloud flavour also

3.[Distributed Computing](#) :

Multiple Machines involved to process a single big Job by contribute their respective compute resources

4. [Features:](#)

Scalability, High Availability, Fault Tolerance, Replication, High Throughput(Read/Writes per second), Low Latency, Parallelism(simultaneous tasks processing by more cores)

5. [Hadoop Eco System:](#)

HDFS: storage, replication(fault-tolerant), master-slave, files split & stored across slaves(dataNodes) as blocks, hadoop v2(default block size: 128Mb)

Map reduce(processing): Map(input as key:value pair) --> Reduce(Map output is processed by trnasformations filter, sort, aggregations...etc)

Sqoop: its an ETL to transfer data from HDFS --> RDBMS(MySql,Hive,Hbase...etc)

Hive(distributed Datawarehouse): Can query by HQL queries

Hbase(NoSQL): Column Oriented DB

Pig(), Yarn(Resource Manager), Spark, Zoo Keeper(wokflows Monitoring/Orchestration)

6. [Types of Processing:](#)

Batch: high volumes of data processed at specific timings at one go, not continuously. here pre-known size before processing data is divided & distributed as blocks among worker nodes as per compute capacity, needs much time & wait until entire data processed to get output

Stream: its continuous data flow needs to process, data size unknown[stock market, social media, e-commerce...etc]

Apache-kafka, spark streaming, amazon kinesis, GCP dataflow, Azure stream analytics...etc

11. Databricks & PySpark(85):

1. [Introduction:](#)

DBFS, dbutils, Mount ADLS, Notebooks, Widgets, jobs, workflows, unity catalogues, delta table...etc

2. [Apache Spark:](#)

Apache-Spark(SuperFast, intermediate read/writes on RAM, intial & Final only on Disk) is alternative to MapReduce(Java based, lengthy,Read/Writes to Disk only). Spark is processing engine/technology used to perform ETL tasks

3. [Spark Components & API:](#)

Spark-Components: Spark-SQL,Spark-Mllib,Spark-Structured Streaming,GraphX

Spark-APIs: Scala,Python,Java,R,SQL

4. [Spark Architecture:](#)

DriverProgram - Spark Context

ClusterManager - It manages cluster nodes resources, driver program requests this for these resources, based on this requests, Cluster Manager launches Executors on Nodes [Standalone, Mesos, Yarn]

WorkerNodes - Executors(Cores --> Tasks)

split & Distribute data across cluster nodes and process it by parallelism

Spark-Application: program written by user, it consists of driver program & executors

DriverProgram: It runs main() function of application and creates sparkcontext, with this context we can create

RDDs(fundamental storage units in spark)

SparkSession introduced as advance of context and it includes SparkContext,SQLContext,HiveContext,StreamContext

Execution-Modes:

ClusterMode(Driver is launched inside cluster)

ClientMode(driver is launched outside cluster i.e on client m/c from where spark app was submitted)

LocalMode(App runs on single m/c)

Executor: it is a Java process launched on worker nodes, these are registered with driver program, adding/removing dynamically during tasks execution

Task: it is a unit/chunk of data distributed to each executor and it runs one or many tasks as per load

Job: process of parallel computation of multiple Tasks

5. [RDD :](#)

Resilient distributed dataset, it is fundamental storage unit in Spark. It can be created using SparkContext, It is immutable(Can't change further). RDD is partitioned across worker Nodes. Resilient means Fault-Tolerant (ability to recover from failures). usually RDDs form from Lists/txt files, Dataframes form from csv, json files
After creating RDD/df, next we can perform operations (Transformations & Actions)

Certain operations require data to be re-distribute across nodes to collect whole relevant data, this triggers called shuffle(data re-copy in wide transformations only in intermediate steps, not in Narrow transformations)
Whenever we use Actions, then only transformations getting real execution, unless they are under DAG logical plan only.
During Actions stage only spark consumes cluster compute power, not in transformations stage

6. Create RDD from List:

```
lst = [1,2,3,4,5] --> print(lst) --> type(lst)
sparkcon = spark.sparkContext --> rdd1 = sparkcon.parallelize(lst) (or) sc.parallelize(lst) --> rdd1.collect()
rdd1.getNumPartitions() [ # by default 8 partitions, we can change it as per our need ]
rdd1.glom().collect()
```

7. Control Partitions in RDD:

```
lst = [1,2,3,4,5] --> rdd1 = sc.parallelize(lst) --> rdd1.collect() --> rdd1.getNumPartitions()
rdd2 = sc.parallelize(lst,5) --> rdd2.getNumPartitions() --> rdd2.glom().collect()
```

8. Create RDD from Txt file:

```
rdd1 = sc.textFile("/dbfs/FilStore/tables/wordcount.txt") --> rdd1.collect() --> rdd1.getNumPartitions()
rdd1 = sc.textFile("/dbfs/FilStore/tables/wordcount.txt",4) --> rdd1.collect() --> rdd1.getNumPartitions()
```

9. Transformations on RDD:

```
rdd1 = sc.textFile("/dbfs/FilStore/tables/wordcount.txt")
rdd2 = rdd1.flatMap(lambda x: x.split(" ")) --> rdd3 = rdd2.map(lambda y: (y,1))
rdd4 = rdd3.reduceByKey(lambda x,y: x+y) --> rdd4.collect()
```

10. Lineage Graph:

rdd1.toDebugString().splitlines() [before doing action, this prepares transformations logical plan]

11. Understanding DAG : Stage wise Graphical view with multiple executed rdd operations. RDD shown as Dots

12. Map Reduce Working:

Input (paragraph/textfile) --> Split(as lines) --> Mapping(words with count line wise) --> Shuffling(grouping same words with total count) --> Reducer(combining same words counting) --> Final(prints processed data as text file)

13. ReduceByKey Vs ReduceByKeyLocally:

```
rdd1 = sc.textFile("/FileStore/tables/001_wordcount.txt")
rdd2 = rdd1.flatMap(lambda x:x.split(" "))
rdd3 = rdd2.map(lambda y:(y,1))
rdd4 = rdd3.reduceByKey(lambda x,y: x+y) --> rdd4.collect() # ReduceByKey gives output as rdd, ReduceByKeyLocally gives
output as dictionary
dict1 = rdd3.reduceByKeyLocally(lambda x,y:x+y) --> print(dict1) # we cant do rdd operations on this being dictionary, it
needs to convert to rdd, next we can do
```

14. GroupBy Key:

```
rdd1 = sc.textFile("/FileStore/tables/001_wordcount.txt")
rdd2 = rdd1.flatMap(lambda x : x.split(" "))
rdd3 = rdd2.map(lambda y : (y,1))
rdd4 = rdd3.reduceByKey(lambda x,y:x+y) --> rdd4.collect()
rdd5 = rdd3.groupByKey().map(lambda x:(x[0],sum(x[1]))) --> rdd5.collect()
rdd5 = rdd3.groupByKey().mapValues(sum) --> rdd5.collect()
```

15. Filter on RDD:

```
rdd1 = sc.textFile("/FileStore/tables/001_wordcount.txt")
rdd2 = rdd1.flatMap(lambda x : x.split(" "))
rdd3 = rdd2.map(lambda y : (y,1))
rdd4 = rdd3.reduceByKey(lambda x,y:x+y) --> rdd4.collect()
rdd5 = rdd4.filter(lambda x: x[0].startswith("h")) --> rdd5.collect()
rdd5 = rdd4.filter(lambda x:x[1] > 30) --> rdd5.collect()
```

16. SortBy, SortByKey on RDD:

```
rdd5 = rdd4.sortByKey() --> rdd5.collect()
rdd5 = rdd4.sortByKey(ascending = False) --> rdd5.collect()
rdd5 = rdd4.sortBy(lambda x:x[1]) --> rdd5.collect()
rdd5 = rdd4.sortBy(lambda x:x[1],ascending = False) --> rdd5.collect()
```

17. Extract Top Bottom from RDD:

```
rdd5 = rdd4.sortBy(lambda x:x[1]).take(5) # Least occuring (Bottom)
rdd5 = rdd4.sortBy(lambda x:x[1],ascending = False).take(5) # Most occuring (Top)
```

18. Save RDD as Txt File:

```
rdd4.saveAsTextFile("/FileStore/tables/output")
rdd1 = sc.textFile("/FileStore/tables/001_wordcount.txt")
```

19. Coalesce and Re-partition on RDD:

```
rdd1 = sc.parallelize([1,2,3,4,5,6,7,8,9,10]) --> rdd1.collect() --> rdd1.glom().collect()
rdd2 = rdd1.coalesce(4) --> rdd2.collect() --> rdd2.getNumPartitions() # by coalesce, we can reduce, but not increase partitions
rdd3 = rdd1.repartition(10) --> rdd3.getNumPartitions() # coalesce is Narrow(No shuffle), but repartition is Wide
Transformation(shuffle)
```

20. Introduction to Accumulators: It's a shared variable [spark driver can assign value to it, worker node can access it]

21. Implementing Accumulators:

```
display(dbutils.fs.ls("/FileStore/tables/"))
display(spark.read.text("dbfs:/FileStore/tables/001_Wordcount.txt"))
rdd = sc.textFile("dbfs:/FileStore/tables/001_Wordcount.txt") --> rdd.collect()
wc = sc.accumulator(0) --> wc.value
def splitandcount(x):
    arr = x.split(" ")
    wc.add(len(arr))
rdd.foreach(splitandcount) --> wc.value # this python function method is recommended
rdd2 = rdd.map(splitandcount) --> rdd2.count() --> wc.value # this sometimes gives wrong values
```

22. BroadCast Variables:

```
data = [(1,"India",10000),(2,"UK",50000),(3,"US",100000),(4,"France",90000)]
rdd = sc.parallelize(data) --> rdd.collect()
products = {1:"Phone",2:"Ipad",3:"TV",4:"Laptop"} --> prodbc = sc.broadcast(products) --> prodbc.value
rdd1 = rdd.map(lambda x:(x[0],prodbc.value[x[0]],x[1],x[2])) --> rdd1.collect() # this broadcast variable used to
combine/merge other dict/list values into first rdd
```

23. **Introduction to df:** its an optimized object in Table format, can easily do many transformations on it.

24. Read CSV in df:

Upload csv file into databricks dbfs storage

```
df = spark.read.csv("/FileStore/tables/Products.csv", header=True, inferSchema=True) --> display(df)
df.printSchema()
df = spark.read.format("csv").option("path", "/FileStore/tables/Products.csv")
.option("header", True).option("inferschema", True).load() --> display(df)
```

25. df Insights:

```
df.count() --> df.columns --> len(df.columns) --> df.printSchema()
```

26. Select Columns in df:

```
df1 = df.select("Order_date", "SalesAmount") --> display(df1)
from pyspark.sql.functions import *
df1 = df.select("Order_date", "SalesAmount", expr("round(SalesAmount*0.05, 2) as VAT")) --> display(df1)
df1 = df.selectExpr("Order_date", "SalesAmount", "round(SalesAmount*0.05, 2) as VAT") --> display(df1)
df1 = df.select(col("OrderDate"), col("SalesAmount"), col("EnglishProductName")) --> display(df1)
```

27. Add Modify Columns in df:

```
from pyspark.sql.functions import *
df = spark.read.format("csv").option("path", "/FileStore/tables/Products.csv").option("header", True).option("inferSchema", True).load()
df1 = df.withColumn("Vat", round(col("SalesAmount")*0.05, 2)) --> display(df1)
df2 = df1.withColumn("CountryOfOrigin", lit("India")) --> display(df2)
df3 = df.withColumns({'Vat': round(col("SalesAmount")*0.05, 2), 'TotalProductCost': col("TotalProductCost").cast("double")})
display(df3)
```

28. Drop Columns in df:

```
df1 = df.drop("Sales", "ProductName") --> display(df1)
```

29. Rename Columns in df:

```
df1 = df.withColumnRenamed("ProductKey", "ProductId").withColumnRenamed("ProducName", "ProductName") --> display(df1)
df1 = df.selectExpr("ProductKey as ProductId", "PoducName as ProductName", "Sales") --> display(df1)
df1 = df.selectExpr("InvoiceNum as InvoiceId").drop("InvoiceNum") --> display(df1)
```

30. Sort Columns in df:

```
df1 = ordersdf.sort("Color", "SalesAmount") --> display(df1)
df1 = ordersdf.sort("Color", "SalesAmount") --> display(df1)
df1 = ordersdf.sort(col("Color"), col("SalesAmount").desc()) --> display(df1)
```

31. Filter data in df:

```
df1 = ordersdf.filter("ProductKey == 310") --> display(df1) --> df1 = ordersdf.filter("ProductKey != 310") --> display(df1)
df1 = ordersdf.filter("SalesAmount > 3000") --> display(df1) --> df1 = ordersdf.filter("Country == 'India'") --> display(df1)
df1 = ordersdf.filter(col("ProductKey") == 310) --> display(df1)
df1 = ordersdf.filter(col("SalesAmount").between(3000, 5000)) --> display(df1)
df1 = ordersdf.filter("ProductKey == 214 and Country == 'Australia'") --> display(df1)
df1 = ordersdf.filter("Country == 'Italy' or Country == 'Australia'") --> display(df1)
df1 = df.where((df.gender=='male') & (df.salary==2000)) --> display(df1)
```

32. Remove Duplicates in df:

```
df1 = empdf.distinct() --> display(df1) --> df1 = empdf.select("city").distinct() --> display(df1)
df1 = empdf.dropDuplicates(["city", "Salary"]) --> display(df1)
```

33. Combine dfs:

```
df3 = empdf1.union(empdf2) --> display(df3) --> df3 = empdf1.union(empdf2).distinct() --> display(df3)
df4 = empdf.unionByName(empnew_df) [ it combine records of 2 tables by matching column names ]
```

34. Pattern Based Filters in df:

```
df = ordersdf.filter(col("ProductName").startswith("A")) --> display(df)
df = ordersdf.filter(col("ProductName").endswith("ge")) --> display(df)
df = ordersdf.filter(col("ProductName").contains("a")) --> display(df)
df = ordersdf.filter(col("ProductName").like("%a%")) --> display(df)
df = ordersdf.filter(col("ProductName").like("A%C%")) --> display(df) # it shows records having many letters b/w A and C
df = ordersdf.filter(col("ProductName").like("A_C%")) --> display(df) # it shows records having only one letter b/w A and C
```

35. Add Column on basis of condition in df:

```
from pyspark.sql.functions import *
df = spark.read.format("csv").option("path", "/FileStore/tables/Products.csv").option("header", True).option("inferSchema", True).load()
df2 = df.withColumn("Tier", when(col("Country")=="US", 1).when(col("Country")=="UK", 2).when(col("Country")=="Germany", 3).otherwise(0))
display(df2)
```

36. Case conversion in df:

```
from pyspark.sql.functions import col, lower, upper, initcap
df = ordersdf.withColumn("ProductName", lower(col("ProductName"))) --> display(df)
df = ordersdf.withColumn("ProductName", upper(col("ProductName"))) --> display(df)
df = ordersdf.withColumn("ProductName", initcap(col("ProductName"))) --> display(df)
```

37. Aggregations in df:

```
from pyspark.sql.functions import *
df = ordersdf.select(sum("SalesAmount").alias("TotalSales")) --> display(df)
df = ordersdf.select(avg("SalesAmount").alias("AvgSales")) --> display(df)
df = ordersdf.select(round(avg("SalesAmount"), 2).alias("AvgSales")) --> display(df) # rounded off to two decimals only
df = ordersdf.select(count("SalesAmount").alias("CountSales")) --> display(df)
df = ordersdf.select(min("SalesAmount").alias("MinSales")) --> display(df)
```

```
df = ordersdf.select(max("SalesAmount").alias("MaxSales")) --> display(df)
df = ordersdf.select(countDistinct("Country").alias("DistinctCountries")) --> display(df)
df = ordersdf.select(sum("SalesAmount").alias("TotalSales"), avg("SalesAmount").alias("AvgSales")) --> display(df)
df1 = ordersdf.selectExpr("sum(SalesAmount) as TotalSales") --> display(df1)
```

38. Aggregations with GroupBy:

```
df = ordersdf.groupBy("Country").sum("SalesAmount") --> display(df)
df = ordersdf.groupBy("Country", "ProductName").sum("SalesAmount") --> display(df)
df = ordersdf.groupBy("Country").sum("SalesAmount", "TaxAmt") --> display(df)
df = ordersdf.groupBy("Country", "ProductName").agg(sum("SalesAmount").alias("TotalSales"), round(avg("SalesAmount"), 2).alias("AvgSales"))
--> display(df)
```

39. Pivot/Un-Pivot in df:

```
pivot_df = ordersdf.groupBy("ProductName").pivot("Country").sum("SalesAmount") --> display(pivot_df)
unpivot_df = pivot_df.unpivot("ProductName", ["Australia", "Canada"], "Country", "Sales") --> display(unpivot_df)
unpDF=pivot_df.unpivot(["ProductKey", "ProductName"], ["Australia", "Canada", "France", "Germany", "India"], "Country", "SalesAmount").drop(na)
display(unpDF)
```

40. Window Functions:

```
Mywin_Asc = Window.orderBy("Salary")
df1 = df.withColumn("Rownum", row_number().over(Mywin)) --> display(df1)

Mywin_desc = Window.orderBy(col("Salary").desc())
df1 = df.withColumn("Rownum", row_number().over(Mywin_desc)) --> display(df1)

Mywin_desc = Window.partitionBy("City", "state").orderBy(col("Salary").desc())
df1 = df.withColumn("Rownum", row_number().over(Mywin_desc)) --> display(df1)

Mywin_desc = Window.orderBy(col("Salary").desc())
df1 = df.withColumn("Ranking", rank().over(Mywin_desc)) --> display(df1)

Mywin_desc = Window.orderBy(col("Salary").desc())
df1 = df.withColumn("Ranking", dense_rank().over(Mywin_desc)) --> display(df1)

Mywin = Window.orderBy(col("Salary")).rowsBetween(Window.unboundedPreceding, Window.currentRow)
df1 = df.withColumn("CumTotal", sum("Salary").over(Mywin)) --> display(df1)

Mywin = Window.orderBy(col("Salary")).rowsBetween(-2, Window.currentRow)
df1 = df.withColumn("CumTotal", sum("Salary").over(Mywin)) --> display(df1)
```

41. Fill Null Values in df:

```
df1 = df.fillna('unknown').fillna(0) --> display(df1)
df1 = df.fillna('unknown', ['Gender']) (or) df.na.fill('unknown', ['Gender']) --> display(df1)
df1 = df.withColumn("id", coalesce("id", lit(0))) --> display(df1)
```

42. Read csv options:

```
df = spark.read.format("csv").option("path", "/FileStore/tables/Employees_comment.csv").option("header", True)
    .option("inferSchema", True).option("delimiter", "|").option("comment", "#").load()
df = spark.read.format("csv").option("path", "/FileStore/tables/SalesMultiline.csv").option("header", True)
    .option("inferSchema", True).option("delimiter", "|").option("multiline", True).load()
df = spark.read.format("csv").option("path", "/FileStore/tables/EmployeesSpace.csv").option("header", True)
    .option("inferSchema", True).option("ignoreLeadingWhiteSpace", True).option("ignoreTrailingWhiteSpace", True).load()
df1 = df.withColumn("collen", len("name")) --> display(df1)
```

43. Impose Schema in df:

```
myschema = "id integer, name string, city string, salary integer"
df = spark.read.format("csv").option("path", "/FileStore/tables/Employees.csv").option("header", True).schema(myschema).load()
from pyspark.sql.types import *
myschema1 = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("city", StringType()),
    StructField("salary", IntegerType())])
df = spark.read.format("csv").option("path", "/FileStore/tables/Employees.csv").option("header", True).schema(myschema1).load()
```

44. Write Modes in df:

```
df.write.format("csv").option("path", "/FileStore/tables/writeModes1/").option("header", True).mode("append").save()
df.write.format("csv").option("path", "/FileStore/tables/writeModes1/").option("header", True).mode("overwrite").save()
df.write.format("csv").option("path", "/FileStore/tables/writeModes1/").option("header", True).mode("ignore").save()
df.write.format("csv").option("path", "/FileStore/tables/Mode1/").option("header", True).mode("errorifexists").save()
```

45. Date Functions:

```
from pyspark.sql.functions import *
df1 = df.withColumn("Today", current_date()).withColumn("CurrentDatetime", current_timestamp()) --> display(df1)
df2 = df1.withColumn("Year", year("Today")).withColumn("Month", month("Today")).withColumn("Day", dayofmonth("Today")) --> display(df2)
df3 = df2.withColumn("Hour", hour("CurrentDatetime")).withColumn("Minute", minute("CurrentDatetime"))
    .withColumn("Second", second("CurrentDatetime")) --> display(df3)
df4 = df3.withColumn("Add10", date_add("Today", 10)) --> display(df4)
df4 = df3.withColumn("Sub10", date_sub("Today", 10)) --> display(df4)
df5 = df4.withColumn("Add_Months", add_months("Today", 5)) --> display(df5)
df6 = df5.withColumn("diff", datediff("Add10", "Today")) --> display(df6)
df7 = df6.withColumn("Today2", date_format("Today", "dd-MMM-yyyy")) --> display(df7)
df7 = df6.withColumn("CurrentDatetime_new", date_format("CurrentDatetime", "dd-MMM-yyyy HH:mm:ss")) --> display(df7)
```

46. Convert RDD to df:

```
lst = [(1, "john", 10000), (2, "Sam", 20000), (3, "Raj", 30000), (4, "Akbar", 40000)] --> print(lst)
rdd1 = sc.parallelize(lst) --> rdd1.collect()
```



```
df1 = rdd1.toDF(["EmpId", "EmpName", "Salary"]) --> display(df1)
rdd2 = df1.rdd --> rdd2.collect()
df2 = spark.createDataFrame(rdd1, ["EmpId", "EmpName", "Salary"]) --> display(df2)
```

47. Explode:

```
from pyspark.sql.types import
from pyspark.sql.functions import
schema = StructType([
    StructField("EnglishProductName", StringType(), True),
    StructField("Color", ArrayType(StringType(), True), True),
    StructField("Description", MapType(StringType(), StringType(), True))])
data = [( 'LL Crankarm', [ 'Red', 'Green', 'Blue' ], { 'Active': 'Yes', 'Price': 1289, 'Qty': 10 },),
        ( 'ML Crankarm', [ 'Red', 'Blue' ], { 'Active': 'Yes', 'Price': 1879, 'Qty': 29 },),
        ( 'HL Crankarm', [ 'Red', 'Green', 'Blue' ], { 'Active': 'Yes', 'Price': 2289, 'Qty': 100 },),
        ( 'Freewheel', [ 'Yellow', 'Green', 'Blue' ], { 'Active': 'No', 'Price': 373, 'Qty': 30 },),
        ( 'Front Derailleur Linkage', [ 'Black', 'Green', 'Blue' ], None)]
df = spark.createDataFrame(data, schema) --> display(df)
df2 = df.select("EnglishProductName", explode("Color").alias(colors)) --> display(df2)
df2 = df.select("EnglishProductName", explode_outer("Color")) --> display(df2)
df2 = df.select("EnglishProductName", explode("Description")) --> display(df2)
df2 = df.select("EnglishProductName", explode_outer("Description")) --> display(df2)
df2 = df.select("EnglishProductName", posexplode("Color")) --> display(df2)
df2 = df.select("EnglishProductName", posexplode_outer("Color")) --> display(df2)
```

48. Working with Array Type Column:

```
df = spark.read.format("csv").option("path", "/FileStore/tables/Products.csv").option("header", True)
    .option("inferSchema", True).load() --> display(df)
df1 = df.select(split("EnglishProductName", " ").alias(ProductName)) --> display(df1)
df2 = df1.selectExpr("ProductName[0]") --> display(df2)
df3 = df1.withColumn("NoOfElements", size("ProductName")).withColumn("Condition", array_contains("ProductName", "Bike")) --> display(df3)
```

49. dbutils:

```
display(dbutils.fs.ls("/FileStore/tables/")) --> dbutils.fs.mkdirs("/FileStore/tables/dev/nn-folder1/")
dbutils.fs.rm("/FileStore/tables/dev/nn-folder1") --> dbutils.fs.rm("/FileStore/tables/nn-1/" recurse=True)
dbutils.fs.cp("/FileStore/tables/nn-1/", "/FileStore/tables/nn-2/", recurse=True)
dbutils.fs.mv("/FileStore/tables/nn-1/", "/FileStore/tables/nn-2/", recurse=True)
```

50. Access ADLS using access keys:

```
# create ADLS and take Access Key from it
# spark.conf.set("fs.azure.account.key.nnadls.dfs.core.windows.net", dbutils.secrets.get(scope="nn-scope1", key=))
spark.conf.set("fs.azure.account.key.nnadls.dfs.core.windows.net", "xxxxxxxxxx")
display(dbutils.fs.ls("abfss://raw@nnadls.dfs.core.windows.net/"))
df = spark.read.format("csv").option("header", True).option("path", "abfss://raw@nnadls.dfs.core.windows.net/orders.csv").load()
display(df)
df.write.format("csv").option("header", True).option("path", "abfss://raw@nnadls.dfs.core.windows.net/output").save()
```

51. Sign up for Azure databricks:

Azure Databricks --> RG, workspace Name, Region, Premium Tier --> create
Create compute cluster --> Create Notebook & write pyspark code

52. Widgets:

```
dbutils.widgets.text("Country", "", "CountryName") --> x = dbutils.widgets.get("Country") --> print(x) --> dbutils.widgets.remove("Country")
dbutils.widgets.combobox("cbocountry", "", ["India", "UK", "US"], "Country-Combo") --> x = dbutils.widgets.get("cbocountry") --> print(x)
dbutils.widgets.dropdown("dpdcountry", "India", ["India", "UK", "US"], "Country-dropdown") --> x = dbutils.widgets.get("dpdcountry") --> print(x)
dbutils.widgets.multiselect("multicountry", "uk", ["India", "uk", "us"], "Country-mselect") --> x = dbutils.widgets.get("multicountry") -->
print(x)
dbutils.widgets.removeAll()
```

53. Calling Other Notebooks: %run /nn/notebook-1 [by run this command in other Notebooks, we can access notebook-1]

54. Read Json File:

```
df = spark.read.format("json").option("path", "/FileStore/tables/Products.json").option("multiline", True).load() --> display(df)
df = df.select("ProductKey", "Detail.ProductName", "Detail.Price", "Detail.SalesAmount") --> display(df)
```

55. Read Multiple Files and Get File Names:

```
from pyspark.sql.functions import input_file_name, split, regexp_replace
df = spark.read.format("csv").option("header", True).option("inferSchema", True).option("path", "/FileStore/tables/Countries/").load()
df1 = df.withColumn("FileName", input_file_name()) --> display(df1)
df1 = df.withColumn("FileName", input_file_name()).select("FileName").distinct() --> display(df1)
df1 = df.withColumn("Countries", regexp_replace(split(input_file_name(), "/")[4], ".csv", "")) --> display(df1)
```

56. Parquet File Format: compressed file(encoded) columnar format, fast read and store data in row groups with metadata

57. Access ADLS using SAS-Token:

```
ADLS --> Shared access signature --> check/tick all Allowed resource types(Service, Container, Object)
and config start, end dates, Next generate, next copy the SAS token
spark.conf.set("fs.azure.account.auth.type.nnadls.dfs.core.windows.net", "SAS")
spark.conf.set("fs.azure.sas.token.provider.type.nnadls.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.sas.FixedSASTokenProvider")
spark.conf.set("fs.azure.sas.fixed.token.nnadls.dfs.core.windows.net", "XXXXXX<sas-token-key>")
display(dbutils.fs.ls("abfss://raw@nnadls.dfs.core.windows.net/"))
dbutils.secrets.listScopes()
Create KeyVault and secret(nnSASToken, xxxxxxxx<sas-token-key>)
Copy databricks workspace URL(https://adb-412983xxxxx.azuredatabricks.net) and in browser
https://adb-412983xxxxx.azuredatabricks.net#secrets/createScope
create Secret Scope(Name(nn-scope1), AllUsers, Vault URI, ResourceID) --> create
spark.conf.set("fs.azure.account.auth.type.nnadls.dfs.core.windows.net", "SAS")
spark.conf.set("fs.azure.sas.token.provider.type.nnadls.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.sas.FixedSASTokenProvider")
spark.conf.set("fs.azure.sas.fixed.token.nnadls.dfs.core.windows.net", dbutils.secrets.get("nn-scope1", "nnSASToken"))
```

In 2nd Notebook, if we need same ADLS access(SASToken), below command
%run <path of Notebook-1>

58. Access ADLS using OAuth:

```
Create App(service principal) in EntraID and Create key Vault with secrets (nn-secret, <xxxEntraID App Client Secret value>), (nn-AppId, <xxxxxAppId>), (nn-tenant,<xxxxTenantId>)
ADLS(IAM) --> blob contributor role to above EntraID App
service_cred = dbutils.secrets.get(scope="nnscope1", key="nn-secret")
tenid = dbutils.secrets.get(scope="nnscope1", key="nn-tenant")
spark.conf.set("fs.azure.account.auth.type.nnnnyctaxisa2025.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.nnnnyctaxisa2025.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.nnnnyctaxisa2025.dfs.core.windows.net",
    dbutils.secrets.get(scope="nnscope1", key="nn-AppId"))
spark.conf.set("fs.azure.account.oauth2.client.secret.nnnnyctaxisa2025.dfs.core.windows.net", f"{service_cred}")
spark.conf.set("fs.azure.account.oauth2.client.endpoint.nnnnyctaxisa2025.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenid}/oauth2/token")
display(dbutils.fs.ls("abfss://bronze@nnnyctaxisa2025.dfs.core.windows.net/"))
```

59. Read & Write Parquet Files :

```
write ADLS accessing code(SAS Token/OAuth...etc) in "nn_access" notebook
# in notebook-1
%run /Workspace/Shared/nn_ws/nn_access
df = spark.read.format("csv").option("header",True).option("inferSchema",True)
    .option("path","abfss://mycontainer@nnadls.dfs.core.windows.net/data/trip.csv").load() --> display(df)
df.repartition(1).write.format("parquet").mode("append").option("path","abfss://mycontainer@nnadls.dfs.core.windows.net/data/").save()
df2 = spark.read.format("parquet")
    .option("path","abfss://mycontainer@nnadls.dfs.core.windows.net/data/tripdata.parquet").load()--> display(df2)
display(spark.read.text("abfss://mycontainer@nnadls.dfs.core.windows.net/data/tripdata.parquet"))# displayed as encoded format
```

60. Delta Format :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.write.format("delta").option("path","/FileStore/tables/DeltaFormat2").save()
df1 = spark.read.format("json").option("path","/FileStore/tables/DeltaFormat2/_delta_log/0000000000000000.json").load() --> display(df1)
df2 = spark.read.format("delta").option("path","/FileStore/tables/DeltaFormat2/").load() --> display(df2) # where
having _delta_log folder & parquet files, that only can be read as delta, for normal folders cannot read as delta
```

61. Create Temp Views :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.createTempView("nnlocalview1") --> %sql Select * from nnlocalview1
df.createOrReplaceTempView("nnlocalview1")
df.createGlobalTempView("nnglobalview1") --> df.createOrReplaceGlobalTempView("nnglobalview1")
%sql Select * from global_temp.nnglobalview1
spark.catalog.listDatabases() --> %sql show databases --> %sql create database nn-db
spark.catalog.listTables() --> %sql show tables
spark.catalog.dropTempView("nnlocalview1") --> spark.catalog.dropGlobalTempView("nnglobalview1")
```

62. Create Managed & Un-Managed Tables :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
# Managed Table from df
df.write.saveAsTable("mgTbl") --> %sql show tables --> %sql describe table mgTbl --> %sql describe detail mgTbl
%sql drop table mgtbl # here table,schema and parquet files & _delta_log total will be deleted during drop
%sql create database nnodb --> df.write.saveAsTable("nnodb.mgTbl")
# Unmanaged/External Table from df
df.write.option("path","/FileStore/tables/Ext/").saveAsTable("extTbl") --> %sql drop table extTbl # here table &
schema only deleted and parquet files & _delta_log will be remain same during drop
# Creating Table from views
df.createTempView("nnview2025") --> %sql create table nnTbl as select * from nnview2025
```

63. Partitioning :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.write.option("path","/FileStore/tables/Countries/").partitionBy("Country").save() # by default delta format
# to read any partitioned parquet files
%sql set spark.databricks.delta.formatCheck.enabled=false
df1 = spark.read.format("parquet").option("path","/FileStore/tables/Countries/Country=France/part-0000-xxxxx.snappy.parquet")
    .load() --> display(df)
df1 = spark.read.format("delta").option("path","/FileStore/tables/Countries/Country=France/").load() --> display(df)
from pyspark.sql.functions import *
dfnew = df.withColumn("Year",year("OrderDate"))
dfnew.write.option("path","/FileStore/tables/Countries/").partitionBy("Country","Year").save()
```

64. Bucketing :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.write.format("csv").option("path","/FileStore/tables/Bkt/").bucketBy(5,"ProductKey").saveAsTable("nnbkt") # here
write in csv, bcoz delta not supported Bucketing
df.write.format("csv").option("path","/FileStore/tables/Bkt/").partitionBy("Country").bucketBy(5,"ProductKey").saveAsTable("nnbkt")
```

65. Control no of Records while writing :

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.write.format("csv").option("header",True).option("path","/FileStore/tables/MaxRecords/").option("maxRecordsPerFile",500).save()
```

66. Max Partition Bytes :

```
spark.conf.set("fs.azure.account.key.nnadls.dfs.core.windows.net","xxxxxxxxxxxxxx<nnadls access key>")
from pyspark.sql.functions import spark_partition_id
spark.conf.set("spark.sql.files.maxPartitionBytes",512*1024*1024) # to convert & each partition size (customized) into Mbs
df = spark.read.format("csv").option("path","abfss://mycontainer@nnadls.dfs.core.windows.net/CSVfiles/tripdata/trips.csv")
    .option("header",True).option("inferSchema",True).load()
display(df) --> df.rdd.getNumPartitions() --> spark.conf.get("spark.sql.files.maxPartitionBytes")
df1 = df.groupBy(spark_partition_id()).count() --> display(df1)
```

67. Spark Query Execution Plans:

```
SQL-Query/df --> Unresolved Logical Plan(Catalog Analysis) --> Logical Plan(Logical Optimization) --> Optimized Logical
Plan(Physical Planning) --> Physical Plans(Cost Model)-->Selected Physical Plan(Code Generation)-->RDDs
df.explain() # it gives only Physical Plan
df.explain(extended = True) # it gives all plans [Parsed Logical Plan, Analysed Logical Plan, Optimized Logical Plan, Physical Plan]
```

68. Joins:

```
Salesdf = spark.read.format("csv").option("path","/FileStore/tables/Sales.csv").option("header",True)
    .option("inferSchema",True).load() --> display(Salesdf)
Productsdf = spark.read.format("csv").option("path","/FileStore/tables/ProductsNew.csv").option("header",True)
    .option("inferSchema",True).load() --> display(Productsdf)
Customersdf = spark.read.format("csv").option("path","/FileStore/tables/Customers.csv").option("header",True)
    .option("inferSchema",True).load() --> display(Customersdf)
Inner_Joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"inner").drop(Salesdf.ProductKey) -->
    display(Inner_Joindf)
Left_Joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"left") --> display(Left_Joindf)
Right_Joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"right") --> display(Right_Joindf)
LeftAnti_Joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"leftanti") --> display(LeftAnti_Joindf)
LeftSemi_Joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"leftsemi") --> display(LeftSemi_Joindf)
multi_joindf = Salesdf.join(Productsdf,Salesdf.ProductKey == Productsdf.ProductKey,"inner").join(Customersdf,Salesdf.CustomerKey ==
    Customersdf.CustomerKey) --> display(multi_joindf)
# If More duplicate ProductIds present in data
SalesMultidf = spark.read.format("csv").option("path","/FileStore/tables/SalesMulti.csv").option("header",True)
    .option("inferSchema",True).load() --> display(SalesMultidf)
ProductsMultidf = spark.read.format("csv").option("path","/FileStore/tables/ProductsMulti.csv").option("header",True)
    .option("inferSchema",True).load() --> display(ProductsMultidf)
joinmulticoldf = SalesMultidf.join(ProductsMultidf,["ProductId","Country"],"inner") --> display(joinmulticoldf) # if both
tables having Joining column Names Same
joinmulticoldf1 = SalesMultidf.join(ProductsMultidf,(SalesMultidf.ProductId == ProductsMultidf.ProductKey) &
    (SalesMultidf.Country == ProductsMultidf.Country),"inner").drop("ProductKey",SalesMultidf.Country)
display(joinmulticoldf1) # here Both Tables Joining Columns names different
```

69. Spark Execution plans for Transformations:

```
from pyspark.sql.functions import *
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df1 = df.filter("Country=India") --> display(df1) --> df1.explain(True)
df2 = df.withColumn("CountryOfOrigin",lit("India")) --> display(df2) --> df2.explain(True)
df3 = df.withColumn("CountryOfOrigin",lit("India")).filter("Country='France'") --> display(df3) --> df3.explain(True)
df4 = df.select(sum(SalesAmount).alias("TotalSales")) --> display(df4) --> df4.explain(True)
df5 = df.groupBy("Country").sum("SalesAmount") --> display(df5) --> df5.explain(True)
CountryDistdf = df.select("Country").distinct() --> display(CountryDistdf) --> CountryDistdf.explain(True)
spark.conf.get("spark.sql.shuffle.partitions") # by default it is 200
spark.conf.set("spark.sql.shuffle.partitions",50) # here shuffles configured to 50
```

70. BroadCast Hash Join Algorithm:

during Joins, df1 is huge large, df2 is very small, then entire df2 data will be copied to all Nodes where df1 spreaded across all Nodes. this broadcast copy done to avoid shuffling. Its upper limit is 8 Gb

```
from pyspark.sql.functions import *
joindf = df.join(ptypedf,df.payment_type == ptypedf.PaymentTypeID) --> display(joindf)
spark.conf.get("spark.sql.autoBroadcastJoinThreshold") --> joindf.explain(True)
spark.conf.get("spark.sql.broadcastTimeout")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold",20*1024*1024) # making from default(10 Mb) --> 20 Mb
spark.conf.set("spark.sql.autoBroadcastJoinThreshold",-1) # to disable broadcast join
joindf = df.join(broadcast(ptypedf),df.payment_type == ptypedf.PaymentTypeID) --> display(joindf)
```

71. Sort Merge Join Algorithm:

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold",20*1024*1024)
spark.conf.get("spark.sql.autoBroadcastJoinThreshold")
spark.conf.get("spark.sql.autoBroadcastJoinThreshold",-1) # it means it disabling BroadCast Join
spark.conf.get("spark.sql.join.preferSortMergeJoin") # by default it is true
joindf.explain(True) # we can observe SMJ by using explain on any join df
```

72. Shuffle Hash Join Algorithm:

```
spark.conf.set("spark.sql.join.preferSortMergeJoin",False) --> spark.conf.get("spark.sql.join.preferSortMergeJoin")
joindf3 = df.join(ptypedf,df.payment_type==ptypedf.PaymentTypeID) --> display(joindf3) --> joindf3.explain(True)
```

73. Sort Merge Bucket Join:

```
df = spark.range(10000000).withColumn("id2",expr("id*2")) --> display(df)
df_copy = df --> display(df_copy)
joindf = df.join(df_copy,df.id == df_copy.id) --> display(joindf) --> joindf.explain(True)
df.write.format("parquet").mode("overwrite").bucketBy(10,"id").saveAsTable("BktIds")
bucketdf1 = spark.table("BktIds")
bucketdf2 = spark.table("BktIds")
joindf1 = df.join(bucketdf1,df.id==bucketdf1.id) --> display(joindf1) --> joindf1.explain(True) # here 1 normal df
(shuffle), 1 bucketed df (its optimized and No shuffle)
```

```
joindf2 = bucketdf1.join(bucketdf2,bucketdf1.id==bucketdf2.id) --> display(joindf2) --> joindf2.explain(True) # here both are bucketed dfs [which column(id) used for bucketing time, same(id) should be used in join also]
```

74. Join Hints:

```
df1 = spark.range(1000000) --> df2 = spark.range(1000000) --> joindf = df1.join(df2,df1.id==df2.id) --> display(joindf)
joindf2 = df1.join(broadcast(df2),df1.id==df2.id) --> display(joindf2) --> joindf2.explain(True)
joindf1 = df1.join(df2.hint("broadcast"),df1.id==df2.id) --> display(joindf1) --> joindf1.explain(True)
joindf1 = df1.join(df2.hint("broadcast"),df1.id>=df2.id) --> display(joindf1) --> joindf1.explain(True) # it applies Broadcast Nested Loop Join
joindf1 = df1.join(df2.hint("merge"),df1.id==df2.id) --> display(joindf1) --> joindf1.explain(True) # it applies SortMergeJoin(SMJ)
joindf1 = df1.join(df2.hint("shuffle_replicate_nl"),df1.id>=df2.id) --> display(joindf1) --> joindf1.explain(True) # its Non-Equal join, it applies Cross Join
```

75. Spark SQL:

Its for processing of structured data by acts as distributed SQL query Engine, can run Most SQL queries in spark env
For querying on df, we need to create TempViews/Managed/Un-Managed Tables, next we can run SQL(DDL/DML) queries on these. and we can save query output as df also.

```
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df.createOrReplaceTempView("nnview1") --> %sql Select * from nnview1 --> _sqlcdf.count() # _sqlcdf is Tempdf to store sparkSQL outputs, for each query this will be overwritten
%sql Select * from nnview1 where Country = 'India' --> _sqlcdf.count()
df1 = spark.sql("Select country,sum(SalesAmount) as TotalSales from nnview1 group by Country") --> display(df1) # this will overcome above _sqlcdf caching problem
df2 = spark.sql("Select country,sum(SalesAmount) as TotalSales from nnview1 where year(OrderByDate)=2021 group by Country") --> display(df1)
df2.write.saveAsTable("dbo.tbl_products") --> %sql Select * from dbo.tbl_products
%sql delete from dbo.tbl_products where InvoiceNum = 'INV100020'
spark.sql("delete from dbo.tbl_products where InvoiceNum = 'INV100020'") --> %sql Select count(*) from dbo.tbl_products
dfcount = spark.sql("update dbo.tbl_products set Country='USA' where Country='United States'") --> display(dfcount)
dfnew = spark.sql("Select * from {tablename} where country = 'India'", tablename=df) --> display(dfnew)
dfnew = spark.sql("Select {col1},{col2} from {tablename} where country = 'India'", col1=df.OrderByDate,col2=df.SalesAmount,tablename=df)
    display(dfnew)
dfnew = spark.sql("Select {col1},sum(SalesAmount) as TotalSales from {tablename} group by {col1}", col1=df.Country, tablename=df) --> display(dfnew)
```

76. Spark Memory management:

Spark developed in Scala which runs in JVM(its VM provides Execution env) to run code. PySpark is API layer added on top of scala to interact with spark via python
All spark APIs underlying execution take place in JVM only. Spark having 2 memories (Execution, Storage Memories), in spark 1.0 whenever these memories fillup fully, it spills/write to disk
To overcome this, in spark 1.6 they combined both execution & Storage as Unified Memory, when this fills up fully, by LRU algorithm storage memory evicts(move) its files to disk
On-HeapMemory(JVM manages it)(ExecutionMemory,StorageMemory,UserMemory,ReservedMemory), Off-Heap memory(OS controls it)(Caching,Overhead (by default 384 Mb for Internal objects))
ExecutionMemory: shuffles,joins,sorts and aggregations
StorageMemory: storing RDDs,df,caching,broadcast variables
UserMemory: UDF,spark metadata
ReservedMemory: by default 300 Mb for Spark Internal objects
On-heap: default fraction is 0.6 Ex: out of spark Job 20 GB Memory split into 20*0.6 = 12 GB (6GB for Execution, 6GB for Storage,7.7GB for UserMemory,300Mb for ReservedMemory)
Off-heap: by default disabled, spark.memory.offheap.enabled = 1, spark.memory.offheap.size = 1 GB

77. Garbage Collection:

GC most imp in JVM(Java) apps. when Memory exhausted, App will crash with memory errors, so at that time spark will pause the app and start the cleaning of unused objects is called GC
GC is automatic free-up/release memory and its 3-step process(Mark --> Sweep --> Compact)
Mark: in this, GC marks reachable(live) & unreachable objects
Sweep: here, it deletes the unreachable objects
Compact: It Compacts Memory by moving around the objects
Serial GC: Its simplest,oldest GC in java. It uses single thread, STW(stop-the-world) approach to pause app during GC, but its decreases throughput for large Apps
Parallel GC: It uses multiple threads and divides heap into smaller regions and perform GC on all at same time, it reduces STW pauses
CMS(Concurrent Mark-Sweep GC): It designed to reduce pause times. It does most of work concurrently
G1 GC: Its low-pause GC. It uses concurrent, parallel phases to maintain high throughput with low pauses.
ZGC collector: Its suitable for very large heap Apps where low latency needed. It uses compacting collector & read-barrier approach
Shenandoah GC: Its also for low latency, minimize pause times for ultra-low by using barrier based approach

78. CPU Terminologies:

CPU Core: It process/executes Tasks, it needs physical core, single core CPU can handle only 1 task at a time
2-core(2 Tasks), 4-core(4 Tasks), 6-core(6 Tasks), 8-core(8 Tasks), 16-core(16 Tasks at a time) Mfg: Qualcomm, Intel, AMD, NVIDIA...etc
Multi-Threading is software code that allows single process to run multiple threads at same time. It used on any CPUs
Hyper-Threading is hardware process that allows single physical core to run multiple threads at same time. It used on supported CPUs
In this trick is there to benefit as Logical processors to doubling the real cores

79. Databricks Compute & Clusters:

Compute is group of computing resources to execute activities
All-Purpose Compute (manual terminate/re-start, single & Multi Nodes, single/multi users)
Job-Compute(compute cluster terminates automatically when Job finished)
Server-less Compute(on-demand,scalable)
SQL Warehouse: To run SQL queries on data objects(rdd/df/datasets)

80. Cluster Manager:

Hadoop Yarn, Apache Mesos, Spark Standalone cluster. with this CM help, it can scale-up 100/1000s of Nodes based on work loads
It helps to launch Executors in Worker Nodes and assign Jobs to respective Nodes
spark-submit.sh, spark-submit.cmd commands available in \$SPARK_HOME/bin directory to execute Jobs/Tasks

spark-submit having some options acts as parameters to customized configure with deploy mode(Cluster Mode, Client Mode), driver memory, executor memory, executor cores...etc

81. Resource Allocation:

Find no of partitions and Get Total no of cores needed to process all these partitions. Next decide Max no of cores per Executor. Next we can get no of executors needed, Calculate memory for each core and for each executor
20 Gb File --> $20480 \text{ Mb} / 128 = 160$ partitions(Tasks) --> 160 Cores needed for Max Parallelism --> 5 Max Cores per Executor recommended --> $32(=160/5)$ Executors Needed for this 160 Tasks
Memory for Each Core (1.5 times reserved memory of 300 Mb) = 450 Mb (Rounded off to 512 Mb)
Memory for each executor = $512 * 5 = 2560 \text{ Mb}$ (2.5 Gb)
Ex: 1 Task takes 4 min time, then for entire 20 Gb how much time (Answer: same 4 min only since Max parallelism all tasks run simultaneously)
Resources Example
Nodes: 10 [Each 16 Cores, 64 Gb RAM -->(1 Core, 1 Gb for OS)] --> 15 Cores, 63 Gb [5 Core per Executor]
 $10 * 5 = 150$ Cores for all Nodes --> Total No of executors --> $150 / 5 = 30$ executors (1 executor reserved for Master Maintenance)
 $29 / 10 \sim 3$ Executors per each worker Node --> $63 \text{ Gb} / 3 = 21 \text{ Gb}$ (Memory per each Executor), but overhead memory consumes 10% means 2.1 Gb wasted each executor
Final Memory allocate for each executor is 18.9 Gb

82. Dynamic Allocation: No content

83. Serialization/De-Serialization:

data stored as objects(rdd,df, datastructures(list,set,dictionary)) which called JVM objects.
When shuffle needed, these objects need to be transfer from one node to another node through Network, but JVM objects cannot transfer directly
first converted into a supported format(byte-stream) which can transfer through Network, this conversion is called Serialization and this also used while data needs to be spilled onto disk
After transfer takes place through Network, again byte stream convert into JVM objects is called Deserialization
Scala using Java-Serialization, but Kryo Serialization(Much Faster) also there another option, default Serialization library in Pyspark is Pyrolite
PySpark using PickleSerializer, but MarshalSerializer(muchfaster) also one another option, but supports fewer data types only

84. Cache & Persist:

```
# Cache & Persist [after using this, further dfs transformations using base dfs from cache/persist only for scanning data for very quick performance]
from pyspark.sql.functions import *
df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
df1 = df.select("OrderDate","InvoiceNum","ProductKey","EnglishProductName","Country","SalesAmount")
df2 = df.1.withColumn("Vat",expr("SalesAmount*0.05 as Vat"))
df3 = df2.filter("Country='India'") --> df3.count()--> df1.cache()--> df1.count() --> df1.is_cached --> df1.unpersist()
from pyspark.storagelevel import StorageLevel
df1.persist(StorageLevel.MEMORY_ONLY) --> df1.count() --> df1.unpersist()
df1.persist(StorageLevel.MEMORY_AND_DISK) --> df1.count() --> df1.unpersist()
df1.persist(StorageLevel.DISK_ONLY) --> df1.count() --> df1.unpersist()
df1.persist(StorageLevel.DISK_ONLY_3) --> df1.count() --> df1.unpersist() # it replicates 3 copies
```

85. Hash Functions:

```
df = spark.read.format("csv").option("path","/FileStore/tables/CustomerInfo.csv").option("header",True)
    .option("inferSchema",True).load() --> display(df)
from pyspark.sql.functions import *
hash_df = df.withColumn("HashKey",hash("EmailAddress","Phone")) --> display(hash_df) # Hash function gives same unique encoded number even we run any times for single/multiple columns
md5hash_df = df.withColumn("HashKey",md5(concat("EmailAddress","Phone"))) --> display(md5hash_df) # It gives hexadecimal unique encoded value[for multiple string columns concat is needed,for single column concat Not needed]
sha1_hash_df = df.withColumn("HashKey",sha1("EmailAddress")) --> display(sha1_hash_df) # it also same as md5, different in bits
sha2_hash_df = df.withColumn("HashKey",sha2("EmailAddress",256)) --> display(sha2_hash_df) # it also same sha1, but here 256 bits used
xxhash_df = df.withColumn("HashKey",xxhash64("EmailAddress")) --> display(xxhash_df) # it uses 64-bit algorithm used
```

13.Databricks(4):

i. **Hive Metastore:** It is default Metastore of databricks.

ii. Introduction to Unity Catalog

It is the only unified and governance solution. We can restrict access against Tables/Schemas/objects for Users/Groups [read-only/write-only...etc] with specifically row/column level permissions.

Unity Catalog is centralized Governing by combining all Metastores, additionally it can store all databricks workspaces, Notebooks, tables, ML Models...etc under it.

Unity Catalog --> [Metastore + User Management] --> [databricks Workspace1,2,3...etc (Notebooks), compute]
Metastore (Metadata) --> Catalogs [Catalog --> Schemas --> Tables/Views/Volumes]

iii. Setting up Unity Catalog

RG, ADLS[nadls, container(raw)], Azure databricks workspace[Name(nnadb2025, premium], databricks access connector, storage blob data contributor Role(ADLS IAM) to access connector

accounts.azure.databricks.net/login --> Catalog --> Create Metastore [Name, Region, ADLS path, access connectorID]

Note: only one Metastore possible each region.

Create Catalog --> click(+) --> add catalog [Name(nncat2025), Type, Storage Location(nnadls), Path(UC)]

Catalog --> External data --> Create Storage credential [Name, access connectorID(ResourceID)] --> create

Catalog --> External data --> Create Ext Location [Name, URL, select above storage credential] --> create

URL: abfss://raw@nnadls.dfs.core.windows.net/

[Test connection]

Next Create compute cluster & Notebook with code.

```
dbutils.fs.ls("abfss://raw@nnadls.dfs.core.windows.net/")
df = spark.read.format("csv").option("header",True).option("path","abfss://raw@nnadls.dfs.core.windows.net/data.csv").load()
display(df)
df.write.format("csv").option("path","abfss://raw@nnadls.dfs.core.windows.net/output/").option("header",True).save()
```

iv. Create Catalog, Schema & Table

Databricks --> Catalog --> click(+) --> add catalog [Name(nncat2025), Type, Storage Location(nnadls), Path(UC)]
nncat2025 --> Create Schema [schema Name(bronze), storage location(nnadls), Path(UC)]

Bronze --> Create Table [browse upload csv file] or through Notebook code as below

```
df = spark.read.format("csv").option("header",True).option("path","abfss://raw@nnadls.dfs.core.windows.net/data.csv").load()
display(df)
df.write.format("delta").saveAsTable("nncat2025.bronze.customers")
```

Note: if we don't give 3 step path in above saveAsTable command, by default it will store in dbfs path. But we specifically given Unity Catalog path(catalog.schema.table), so Table will be stored under Unity Catalog and its relevant delta_log folder and parquet files will be saved under path in adls (raw/UC/_Unitystorage/schemas/xxxxxx/tables/xxxxx/_delta_log & part-xxxx.parquet

14. Spark Structured Streaming(9):

1. Introduction:

2. ReadStream:

```
dbutils.fs.mkdirs("/FileStore/tables/StreamingSource/",recurse=True) # Later in this folder we need to upload
files one by one to check streaming
from pyspark.sql.types import *
emp_schema = StructType([
    StructField("id",LongType(),False),
    StructField("name",StringType(),False),
    StructField("city",StringType(),False),
    StructField("salary",IntegerType(),False)])
dstr = spark.readStream.format("csv").option("path","/FileStore/tables/StreamingSource/").schema(emp_schema)
.option("header",True).load() --> type(dstr) --> dstr.isStreaming --> display(dstr)
```

3. Write Stream:

```
source_path = "/FileStore/tables/StreamingSource/"
sink_path = "/FileStore/tables/StreamingSink/"
checkpoint_path = "/FileStore/tables/StreamingCheckPoint/"
# dbutils.fs.rm(source_path,recurse=True) --> dbutils.fs.rm(sink_path,recurse=True) --> dbutils.fs.rm(checkpoint_path,recurse=True)
dbutils.fs.mkdirs(source_path) --> dbutils.fs.mkdirs(sink_path) --> dbutils.fs.mkdirs(checkpoint_path)
from pyspark.sql.types import *
emp_schema = StructType([
    StructField("id",LongType(),False),
    StructField("name",StringType(),False),
    StructField("city",StringType(),False),
    StructField("salary",IntegerType(),False)])
dstr = spark.readStream.format("csv").option("path",source_path).schema(emp_schema)
.option("header",True).load() --> display(dstr)
dstr.writeStream.format("csv").option("header",True).option("path",sink_path)
.option("checkpointLocation",checkpoint_path).start().awaitTermination()
```

4. Stream OutPutModes:

```
dstr = spark.readStream.format("csv").option("path","/FileStore/tables/StreamingSource/").schema(emp_schema)
.option("header",True).load() --> type(dstr) --> dstr.isStreaming --> display(dstr)
aggdf = dstr.groupBy("product").count()
aggdf.writeStream.format("console").outputMode("complete").start()
aggdf.writeStream.format("console").outputMode("update").start()
aggdf.writeStream.format("console").outputMode("append").start()
```

5. Stream Sources & Sinks:

```
# Sources & Sinks
# Rate
ratedf = spark.readStream.format("rate").load() --> display(ratedf) # it gives timestamp and value columns to test sparkstream
# File
from pyspark.sql.types import *
emp_schema = StructType([
    StructField("id",LongType(),False),
    StructField("name",StringType(),False),
    StructField("city",StringType(),False),
    StructField("salary",IntegerType(),False)])
fileddf = spark.readStream.format("csv").option("path","/FileStore/tables/StreamingSource/").schema(emp_schema)
.option("header",True).load() --> display(fileddf)
# Socket
socketddf = spark.readStream.format("socket").option("host","<IP address>").option("port","<port no>").load() --> display(socketddf)
# Kafka
kafkadf = spark.readStream.format("kafka").option("kafka.bootstrap.servers","<server url:portnumber>")
.option("subscribe","input-topic").load() --> display(kafkadf)
# Sinks-Files
fileddf.writeStream.outputMode("append").option("path","/FileStore/table/StreamingSink/").option("format","csv")
.option("checkpointLocation","/FileStore/table/StreamingCheckPoint/").option("header",True).start().awaitTermination()
```

```
# Kafka Sink
df.writeStream.format("kafka").option("kafka.bootstrap.servers", "<server url:portnumber>").option("topic", "products")
  .option("checkpointLocation", "/FileStore/table/StreamingCheckpoint").start().awaitTermination()

# Console Sink
aggdf.writeStream.format("console").outputMode("update").start()
```

6. Stream Triggers:

```
# Triggers
readdf = spark.readStream.format("csv").option("path", source_path).schema(orders_schema).option("maxFilesPerTrigger", 1)
  .option("header", True).load()
aggdf = readdf.groupBy("product").agg(count("product").alias("TotalCount").withColumn("Timestamp", current_timestamp()))
aggdf.writeStream.format("console").outputMode("complete").trigger(processingTime="30 seconds")
  .option("truncate", False).start()
aggdf.writeStream.format("console").outputMode("complete").trigger(availableNow=True).option("truncate", False).start()
```

7. Stream Joins:

```
salesdf = spark.read.format("csv").option("path", "/FileStore/tables/Sales.csv").option("header", True)
  .option("inferSchema", True).load() --> display(salesdf)
from pyspark.sql.types import *
products_schema = StructType([
    StructField("ProductKey", LongType(), False),
    StructField("EnglishProductName", StringType(), False)])
dbutils.fs.mkdirs("/FileStore/tables/ProductsStreaming")
productsdf = spark.readStream.format("csv").option("path", "/FileStore/tables/ProductsStreaming/").option("header", True)
  .schema(products_schema).load() --> display(productsdf)
joindf = salesdf.join(productsdf, "ProductKey", "inner") --> display(joindf) # output: success, result will come
joindf = salesdf.join(productsdf, "ProductKey", "left") --> display(joindf) # batchdf left, streamdf right, So gives Error
# result not comes
joindf = salesdf.join(productsdf, "ProductKey", "right") --> display(joindf) # output success, result will come
joindf = salesdf.join(productsdf, "ProductKey", "full_outer") --> display(joindf) # it also gives Error, result not comes
```

8. Stateful Vs Stateless:

9. Window Operations:

12. DeltaLake & Lakehouse(19):

1. History of Data Architectures:

SQL-RDBMS(1970-80) --> data-warehouse(1980s) --> DataLake(HDFS/ADLS/S3/GCS)(2010s) --> Data Lakehouse(2020s)

2. Introduction of delta Lake :

datawarehouse: Its central repo concept by collecting historical data from multiple sources and provided an single integrated view for businesses to analyse easily(filters, aggregates...etc)

It includes mainly fact & dimension Tables and schema concepts and it supports structured data only

Datalake: It also central repo supports structured, semi-structured, un-structured data

DataLakehouse: It is built on top of datalake by extracting metadata and having both properties of datalake + datawarehouse. here it eliminates copy/load of entire data from datalake --> datawarehouse

here we can query SQL queries direct on datalake files, it saves more cost & time.

Low cost, ACID compliance, data versioning, optimized querying...etc benefits

DeltaLake: It is an open table format. with this abstraction layer concept(extracting metadata) only, we can make Lakehouse architecture with Delta Tables on top of dataLake

ACID compliance, data versioning, SCD, CDC (merge, upsert, delete)

3. Read/Write Delta Format :

```
df = spark.range(1,11) --> df.write.format("delta").save("/FileStore/tables/DeltaTables/nntbl") # output is file format. here we
# can use dbfs/adls csv, parquet files also instead of range
df1 = spark.read.format("delta").load("/FileStore/tables/DeltaTables/nntbl") # with above command, actual data will be in parquet
# file, metadata in _delta_log folder(json, crc files)
df.write.saveAsTable("nntbl") --> %sql Select * from nntbl # here output is table format
```

4. Understanding Delta Log :

```
df = spark.range(1,11) --> df.write.format("delta").save("/FileStore/tables/DeltaTables/nntbl")
dflog = spark.read.format("json").option("path", "/FileStore/tables/DeltaTables/nntbl/_delta_log/xxxx0000.json").load()
%sql update delta.`/FileStore/tables/DeltaTables/nntbl` set id=20 where id=5
%sql delete from delta.`/FileStore/tables/DeltaTables/nntbl` where id=3
%sql select * from delta.`/FileStore/tables/DeltaTables/nntbl` # this command to query delta Lake file formats when not saved as
# table(saveAsTable option)
%sql describe history delta.`/FileStore/tables/DeltaTables/nntbl`
from delta.tables import * --> from pyspark.sql.functions import *
dbtable = DeltaTable.forPath(spark, "/FileStore/tables/DeltaTables/nntbl") # output: delta table
# update, delete of delta table thru pyspark
dbtable.update(condition="id=6", set={"id": "60"})
dbtable.delete(condition="id=7")
%sql select * from delta.`/FileStore/tables/DeltaTables/nntbl`
```

5. Version History & Time Travel :

```
df = spark.read.format("csv").option("path", "/FileStore/tables/Products.csv").option("header", True).option("inferSchema", True).load()
df.write.format("delta").saveAsTable("tblproducts") --> %sql describe history tblproducts --> %sql describe table extended tblproducts
%sql update tblproducts set Country='USA' where Country='United States' --> %sql delete from tblproducts where Country='USA'
# time travel view(sql method)
%sql select * from tblproducts --> %sql select * from tblproducts version as of 1
%sql select * from tblproducts timestamp as of '2024-06-16T18:19:48.000+0000'
# time travel view(pyspark method)
df = spark.read.format("delta").option("versionAsOf", 0).load("/user/hive/warehouse/tblproducts")
df = spark.read.format("delta").option("timeStampAsOf", "2024-06-16T18:19:48.000+0000").load("/user/hive/warehouse/tblproducts")
# restore version
# restore version thru SQL
%sql restore tblproducts to version as of 1
```

```
%sql restore tblproducts to timestamp as of '2024-06-16T18:19:48.000+0000'
# restore version thru PySpark
dtbl = DeltaTable.forName(spark,"tblproducts") --> dtbl.restoreToVersion(2) --> dtbl.restoreToTimestamp("2024-06-16T18:19:48.000+0000")
```

6. Check Point:

```
# Check Point
schema = "ProductId : int, ProductName : string"
df = spark.createDataFrame([(1,'Adjustable Race')],schema) --> display(df)
df.write.format("delta").saveAsTable("products")
lst = ['BearingBall','BB Ball Bearing','Headset BallBearings','Blade','LL Crankarm','ML Crankarm','HL Crankarm','Chainring
bolts','Chainring Nut']
for i in range(2,11):
    df = spark.createDataFrame([(i,list[i-2])],schema)
    df.write.format("delta").mode("append").saveAsTable("products")
df = spark.createDataFrame([(11,'Chainring')],schema)
df.write.format('delta').mode('append').saveAsTable('products') # after every 10th file, 1 checkpoint.parquet file is creating and
it consists its previous 10 files metadata
%sql alter table products set tblproperties ("delta.checkpointInterval" = "20") # it creates checkpoint files after every 20th file
```

7. Create Delta Tables :

```
# thru SQL
%sql create database if not exists nndb --> %sql create table if not exists nndb.tblproducts(ProductId int, ProductName string) using
delta
%sql create table if not exists nndb.tblproducts(ProductId int, ProductName string) using delta location '/FileStore/tables/products1/'
# thru pyspark
dfproducts.write.format("delta").saveAsTable("tblproducts") # products.csv file --> dfproducts --> delta table(tblproducts)
DeltaTable.createIfNotExists(spark).tableName("nndb.tblproducts").addColumn("ProductId","int").addColumn("ProductName","string").addCol
umn("Price","double").execute()
%sql describe table extended nndb.tblproducts
```

8. Generated Columns :

```
%sql create table nndb.Employees(
    EmpId int, EmpName string, DOJ timestamp,
    YearOfJoining int generated always as (year(DOJ)),
    MonthOfJoining int generated always as (month(DOJ)),
    DayOfJoining int generated always as (day(DOJ))
) using delta
%sql describe formatted nndb.Employees
%sql insert into nndb.Employees(EmpId,EmpName,DOJ) values(1,'John','2020-01-12'),(2,'Sam','2021-06-11'),(3,'Aman','2022-09-06')
%sql select * from nndb.Employees
```

9. Create Partitioned Table :

```
%sql create table nndb.Products1(
    OrderDate date, Invoicenum string, ProductKey int, Country string, EnglishProductName string, SalesAmount double, UnitPrice double,
    OrderQty int, TaxAmt double, TotalProductCost double,
    OrderYear int generated always as (year(OrderDate))
) using delta partitioned by (Country,OrderYear)
dfproducts.write.format("delta").mode("append").saveAsTable("nndb.Products1") # to append data into any existed Delta Table thru
pyspark
```

10. Schema Evolution :

```
dfproducts.write.format("parquet").option("path","/FileStore/tables/ParquetSchema").save()
dfnew = spark.createDataFrame([(0.05,0.02),(0.05,0.01),(0.04,0.02),(0.05,0.01)],["VatPercent","DiscountPercent"])
dfnew.write.format("parquet").mode("append").option("path","/Filestore/tables/ParquetSchema").save()
df_products_parquet = spark.read.format("parquet").option("path","/FileStore/tables/ParquetSchema").load() # in parquet schema
evolution not possible
dfproducts.write.format("delta").option("path","/FileStore/tables/DeltaSchema").save()
dfnew.write.format("delta").mode("append").option("mergeSchema",True).option("path","/FileStore/tables/DeltaSchema").save() # in
delta schema evolution possible
dfnew.write.format("delta").mode("overwrite").option("mergeSchema",True).option("path","/FileStore/tables/DeltaSchema").save() # dont
use overwrite in schema evolution, it erases total data of left side(1st table), only 2nd table data remain
df_delta_products = spark.read.format("delta").option("path","/FileStore/tables/DeltaSchema").load()
spark.conf.get("spark.databricks.delta.schema.autoMerge.enabled")
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled",True)
```

11. Copy Into :

```
% sql create table tbl_employees(id int, name string, city string, salary double) usinf delta
%sql copy into tbl_employees from
    (Select id::int, name::string, city::string, salary:double from 'dbfs:/FileStore/tables/Employees.csv')
    fileformat = CSV format_options("header"="true")
%sql select * from tbl_employees
# Note: the above copy command cannot allow 2nd time same file copying
%sql create table tbl_employees_new using delta
%sql copy into tbl_employees_new from 'dbfs:/FileStore/tables/Employees.csv'
    fileformat = CSV format options('inferSchema'='true', 'mergeSchema'='true', 'header'='true') copy_options('mergeSchema'='true')
```

12. Merge :

```
%sql create tbl_Employees(id int, name string, city string, salary double) using delta
%sql create tbl_Employees_stage(id int, name string, city string, salary double) using delta
%sql copy into tbl_Employees_stage from
    (Select id::int, name::string, city::string, salary:double from 'dbfs:/FileStore/tables/Employees.csv') fileformat = CSV
format_options("header"="true")
%sql delete from tbl_Employees where id>5
%sql update tbl_Employees set salary=5000 where id=5
%sql select * from tbl_Employees --> %sql select * from tbl_Employees_stage
%sql merge into tbl_Employees as T using tbl_Employees_stage as S on T.id = S.id
```



```

when matched then
  update set T.city = S.city, T.Salary = S.Salary
when not matched then
  insert (id,Name,city,Salary) values(id,Name,city,Salary)
when not matched by source then
  delete

```

13. Column Statistics :

```

df.write.format("delta").option("path","/FileStore/tables/ColumnStatistics").saveAsTable("tblcolstats")
%sql alter table tblcolstats set tblproperties('delta.dataSkippingNumIndexedCols' = 40);
df.write.format("delta").mode("overwrite").option("path","/FileStore/tables/ColumnStatistics").saveAsTable("tblcolstats")

```

14. Optimize-Vacuum-Zorder :

```

df = spark.read.format("csv").option("path","/FileStore/tables/Products.csv").option("header",True).option("inferSchema",True).load() -> display(df)
df.repartition(4).write.format("delta").saveAsTable("tblperformance") --> %sql describe history tblperformance
%sql update tblperformance set Country = 'USA' where Country = 'United States'
spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled","false")
%sql vacuum tblperformance retain 0 hours dry run
%sql optimize tblperformance --> %sql vacuum tblperformance retain 0 hours --> --> %sql describe history tblperformance
spark.conf.get("spark.databricks.delta.optimize.maxFileSize")
from delta.tables import *
deltaTable = DeltaTable.forName(spark,"tblPerformance")
deltaTable.logRetentionDuration = "interval 40 days"
deltaTable.deletedFileRetentionDuration = "interval 10 days"
df.repartition(4).write.format("delta").partitionBy("Country").saveAsTable("tblperformance1")
%sql optimize tblperformance1 where Country = "India" --> %sql vacuum tblperformance1 retain 0 hours
df.repartition(7).write.format("delta").saveAsTable("tblperformanceZ")
%sql optimize tblperformanceZ zorder by (Country) --> %sql vacuum tblperformanceZ retain 0 hours

```

15. Liquid Clustering :

```

%sql create table tblEmployees_new(orderDate timestamp, InvoiceNum string, Country string, Salesamount double) cluster by (Country);
%sql create table tblEmployees_new1(orderDate timestamp, InvoiceNum string, Country string, Salesamount double)
%sql alter table tblEmployees_new1 cluster by (Country) # to add clustering to existed table
df.select(col("OrderDate").cast("timestamp"),"Invoicenum","Country","SalesAmount").write.mode("append").saveAsTable("tblEmployees_new")
%sql optimize tblEmployees_new # to activate liquid clustering
%sql alter table tblEmployees_new1 cluster by none # to remove liquid clustering

```

16. Change Data Feed :

```

df.write.saveAsTable("tbl_Employee") --> %sql describe detail tbl_Employee
%sql alter table tbl_Employee set tblproperties (delta.enableChangeDataFeed = true) # it creates CDF folder in delta_log folder and it shows update, delete query logs only, but not show insert query logs
%sql update tbl_Employee set salary=50000 where id=7
%sql describe history tbl_Employee
%sql select * from table_changes('tbl_Employee',2,4) # it shows CDF logs(delete/update/write) b/w 2 & 4 versions
%sql select * from table_changes('tbl_Employee','2024-10-04T14:15xxxx','2024-10-04T14:20xxxx') # it shows CDF logs(delete/update/write) b/w two timestamps
cdf_df =
spark.read.format("delta").option('readChangeFeed','true').option('startingVersion',2).option('endingVersion',4).table(tbl_Employee)
display(cdf_df)
%sql create table if not exists tbl_Employees2(id int, Name string, city string, salary double)
tblproperties(delta.enableChangeDataFeed = true);
%sql set spark.databricks.delta.properties.defaults.enableChangeFeed = true

```

17. Reorg Table :

```

df.write.saveAsTable("employees")
%sql alter table employees set tblproperties('delta.columnMapping.mode' = 'name', 'delta.minReaderVersion' = '2', 'delta.minWriterVersion' = '5')
%sql alter table employees drop column city
%sql reorg table employees apply(purge)
%sql set spark.databricks.delta.retentionDurationCheck.enabled = false
%sql vacuum employees retain 0 hours
%sql select * from employee

```

18. Deletion Vectors :

```

df = spark.range(100000000) --> df.write.saveAsTable("tblids") --> %sql show tblproperties tblids --> %sql describe detail tblids
%sql alter table tblids set tblproperties('delta.enableDeletionVectors' = true) --> %sql delete from tblids where id = 182638
%sql reorg table tblids appy(purge) --> %sql set spark.databricks.delta.retentionDurationCheck.enabled = false --> %sql vacuum tblids retain 0 hours
%sql delete from tblids where id in(253420,83535,172632) --> %sql reorg table tblids appy(purge) --> %sql vacuum tblids retain 0 hours

```

19. SCD-Type1 :

```

empdf.write.saveAsTable("tblemployee")
empupdates_df =
spark.read.format("csv").option("header",True).option("inferSchema",True).option("path","/FileStore/tables/EmployeesUpdate.csv").load()
empupdates_df.write.saveAsTable("tblemployee_stage")
%sql select * from tblemployee_stage
except
Select * from tblemployee
%sql merge into tblemployee as T using tblemployee_stage as S on T.id = S.id
  when matched then
    update set T.Name = S.Name, T.city = S.city, T.Salary = S.Salary
  when not matched then
    insert (id,Name,city,Salary) values(S.id,S.Name,S.city,S.Salary)
%sql select * from tblemployee
# thru spark

```

```
from delta.tables import *
empdf.write.saveAsTable("tblemployee")          # this table is creating from empdf
employeeTable = DeltaTable.forName(spark,"tblemployee")
employeeTable.alias("Target").merge(source = empupdates_df.alias("Source"), condition="Target.id=Source.id")
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
(employeeTable.alias("Target").merge(source = empupdates_df.alias("Source"), condition="Target.id=Source.id")
    .whenMatchedUpdate(
        set={
            "city":"Source.city",
            "Salary":"Source.Salary"
        })
    .whenNotMatchedInsert(
        values={
            "id":"Source.id",
            "Name":"Source.Name",
            "city":"Source.city",
            "Salary":"Source.Salary"
        })).execute())
%sql select * from tblemployee
```

Raja's databricks & Pyspark:

1. Delta Table :

```
# Delta Table
# Method-1 (using Pyspark by direct creating table)
from delta.tables import *
DeltaTable.create(spark).tableName("nn_tb1").addColumn("emp_id", "int") \
# here we can use nn_tb1 or
dbo.nn_tb1 or <custom schema name>.nn_tb1
.addColumn("emp_name", "string").addColumn("gender", "string") \
.addColumn("salary", "int").addColumn("Dept", "string") \
.property("description", "Table created for trial") \
.location("/FileStore/tables/delta/nn_delta/") \
# we can use DBFS or ADLS or any
storage location and also can use createIfNotExists(spark) or createOrReplace(spark) instead of create(spark)
.execute()

# Note: in above, in databricks in database Tables nn_tb1 delta table will be created and
# simultaneously in DBFS file system in above said path, delta log folder(.crc, .json files) will be created,
# whenever we insert data into table, parquet files will be created along with delta log folder path
%fs ls /FileStore/tables/delta/nn_delta/_delta_log
%fs head /FileStore/tables/delta/nn_delta/_delta_log/0*****01.json
display(spark.read.format("delta").load("/FileStore/tables/delta/nn_delta/"))
%sql
insert into nn_tb1 values(100,"Stephen","M",2000,"IT");
insert into nn_tb1 values(200,"Philip","M",8000,"HR");
insert into nn_tb1 values(300,"Lara","F",6000,"Sales");
select * from nn_tb1;
delete from nn_tb1 where emp_id=100;
%fs ls /FileStore/tables/delta/nn_delta/_delta_log
# Method-2 (SQL type)
%sql
create table nn_emp(emp_id int, emp_Name string, gender string, salary int, dept string) using delta
create table if not exists nn_emp(emp_id int, emp_Name string, gender string, salary int, dept string) using delta
create or replace table if not exists nn_emp(emp_id int, emp_Name string, gender string, salary int, dept string)
using delta location '/FileStore/tables/delta/nn_delta2025' # s3/gcs/adls/dbfs any storage
# Method-3 (using dataframe)
df = spark.createDataFrame(data=emp_data, schema=emp_schema) --> display(df)
df.write.format("delta").saveAsTable("nn_emp_tb1")
df.write.format("delta").saveAsTable("dbo.nn_emp_tb1") (or) df.write.format("delta").saveAsTable("SalesLT.nn_table1")
df.write.format("delta").mode("append/overwrite").saveAsTable("nn_emp_tb1")
df.write.insertInto('nn_emp_tb1', overwrite = False)
%sql
select * from nn_emp_tb1 (or) select * from SalesLT.nn_table1
display(spark.sql("select * from nn_emp_tb1"))
```

2. Delta Table Instance

```
# Note: For each Delta Table, multiple delta table instances[soft-link(replica) of actual delta table] are developed.
# we need to perform DML queries on these instances by pyspark instead of SQL, if we delete any records in original
delta table, in Instance also same will be reflected and vice versa
DeltaTable.forPath(spark, "<Delta Table path>")
DeltaTable.forName(spark, "<Delta Table Name>")
deltaInstance1 = DeltaTable.forPath(spark, "/FileStore/tables/delta/nn_emp_tb1") --> display(deltaInstance1.toDF())
deltaInstance1.delete("emp_id = 200") # its pyspark delete command similar to delete query in SQL
deltaInstance2 = DeltaTable.forName(spark, "nn_emp_tb1") --> display(deltaInstance2.toDF())
%sql describe history nn_emp_tb1; # in SQL
display(deltaInstance2.history()) [ in pyspark]
```

3. TempViews from df

```
# dataframe into temp view
df = spark.createDataFrame(data=emp_data, schema=emp_schema) --> display(df)
df.createOrReplaceTempView("nn_view") --> %sql select * from nn_view
%sql insert into nn_emp_tb2 select * from nn_view (or) spark.sql("insert into nn_emp_tb2 select * from nn-view")
```

4. Delete Delta Table

```
# delete delta table
%sql delete from nn_emp_tb2 where emp_id = 100
%sql delete from delta.`/FileStore/tables/delta/nn_emp_tb1` where emp_id = 200
spark.sql("delete from nn_emp_tb1 where emp_id = 300")
# delete delta table Instance Method
deltaInstance1 = DeltaTable.forName(spark, 'nn_emp_tb3')
deltaInstance1.delete("emp_id = 400") (or) deltaInstance1.delete(col('emp_id') == 400)
deltaInstance1.delete("emp_id = 500 and gender = 'F'")
```

5. Update Delta Table

```
# update delta table
%sql update nn_emp_tb1 set salary = 10000 where emp_name = 'Mark';
%sql update delta.`/FileStore/tables/delta/nn_emp_tb1` set salary = 12000 where emp_name = 'Mark';
# update delta table Instance Method
deltaInstance1 = DeltaTable.forPath(spark, "/FileStore/tables/delta/nn_emp_tb1")
deltaInstance1.update(condition = "emp_name = 'Mark'", set = {"salary": "15000"})
deltaInstance1.update(condition = col("emp_name") == 'Mark', set = {'Dept': lit('IT')})
```

6. SCD-1 (Merge)

```
# SCD-1 (Merge)
create dataframe from data, schema and delta table(nn_dim_tb1) similar to as above
df.createOrReplaceTempView("nn_view") # here source is nn_view Target is nn_emp_tb1
%sql merge into nn_dim_tb1 as target
using nn_view as source on target.emp_id = source.emp_id
when Matched then
update set target.name = source.name, target.city = source.city, target.country = source.country, target.contact_no = source.contact_no
when not matched then
insert( emp_id, name, city, country, contact_no) values(emp_id, name, city, country, contact_no)
# SCD-1 (Merge) pyspark Method
create dataframe from data, schema
delta_Instance = DeltaTable.forPath(spark, "/FileStore/tables/delta_merge")
delta_Instance.alias("target").merge(source = df.alias("source"), condition = "target.emp_id = source.emp_id")
    .whenMatchedUpdate(set =
    {
        "name": "source.name",
        "city": "source.city",
        "country": "source.country",
        "contact_no": "source.contact_no"
    })
    .whenNotMatchedInsert(values =
    {
        "emp_id": "source.emp_id",
        "name": "source.name",
        "city": "source.city",
        "country": "source.country",
        "contact_no": "source.contact_no"
    })
    .execute()
```

7. SCD-2

```
# SCD-2 [ Change of attribute/value of entities over a period of time ]
SCD-1: overwrite, history of data/records changes not maintained
SCD-2: history of data/records changes maintained as version type with start and end dates, latest only be active Y-Flag, remain old
changes of specific record are in-active N-Flag.
In aws S3 & github repo also, bkt objects having versioning facility similar to SCD-2 type
SCD-3: history of data/records changes maintained in separate column
Create Delta Table(source) with inserting some records, create delta table instance (for target)
from delta import *
targetTable = DeltaTable.forPath(spark, "/FileStore/tables/scd2demo/")
target_df = targetTable.toDF() --> display(target_df)
create data, schema and dataframe
join_df = source_df.join(target_df, (source_df.pk1 == target_df.pk1) & (source_df.pk2 == target_df.pk2)) &
    (target_df.active_status == 'Y'), "leftouter").select(source_df["*"], target_df.pk1.alias("target_pk1"),
    target_df.pk2.alias("target_pk2"), target_df.dim1.alias("target_dim1"), target_df.dim2.alias("target_dim2"),
    target_df.dim3.alias("target_dim3"), target_df.dim4.alias("target_dim4")) --> display(join_df)
filter_df = join_df.filter(xxhash64( join_df.dim1, join_df.dim2, join_df.dim3, join_df.dim4) != xxhash64( join_df.target_dim1,
    join_df.target_dim2, join_df.target_dim3, join_df.target_dim4)) --> display(filter_df)
merge_df = filter_df.withColumn("MergeKey", concat(filter_df.pk1, filter_df.pk2)) --> display(merge_df)
dummy_df = filter_df.filter("target_pk1 is not null").withColumn("MergeKey", lit(None)) --> display(dummy_df)
scd_df = merge_df.union(dummy_df) --> display(scd_df)
targetTable.alias("target").merge(source = source_df.alias("source"), condition = "concat(target.pk1, target.pk2) = source.MergeKey and
target.active_status = 'Y' ")
    .whenMatchedUpdate(set =
    {
        "active_status": "'N'"
        "end_date": "current_date"
    })
    .whenNotMatchedInsert(values =
    {
        "pk1": "source.pk1",
        "pk2": "source.pk2",
        "dim1": "source.dim1",
        "dim2": "source.dim2",
        "dim3": "source.dim3",
        "dim4": "source.dim4",
        "active_status": "'Y'",
        "start_date": "current_date",
        "end_date": "to_date('9999-12-31','yyyy-MM-dd')""
    })
    .execute()
%sql select * from scd2demo
```

8. Time Travel in Delta Tables

```
# Time Travel in Delta tables
%sql describe history scd2demo
# Methods using PySpark
df = spark.read.format("delta").option("timestampAsOf", "2022-05-15T09:55:53.000+0000").table("scd2demo") --> display(df)
df = spark.read.format("delta").option("timestampAsOf", "2022-05-15T09:55:53.000+0000").load("/FileStore/tables/scd2demo")
display(df)
df = spark.read.format("delta").option("versionAsOf", 2).load("/FileStore/tables/scd2demo") --> display(df)
df = spark.read.format("delta").option("versionAsOf", 2).table("scd2demo") --> display(df)
# Methods using SQL
%sql select * from scd2demo version as of 2
%sql select * from delta.`/FileStore/tables/scd2demo` version as of 2
```



```
%sql select * from scd2demo timestamp as of "2022-05-15T09:55:53.000+0000"
%sql select * delta.`/FileStore/tables/scd2demo` timestamp as of "2022-05-15T09:55:53.000+0000"
```

9. Restore in Delta Tables

```
# Restore in Delta Tables
Create Delta Table(source) with inserting some records, create delta table instance(for target) as above
display(targetTable.history())
targetTable.restoreToVersion(6) --> %sql select * from scd2demo      # it restores the 6th version of Delta Table
targetTable.restoreToTimestamp("2022-05-16T07:34:50.000+0000")      # it restores to previos specific timestamp of Delta Table
```

10. Optimize in Delta Tables

```
# Optimise in Delta Tables
# Create Delta Table(source) with inserting some records, create delta table instance(for target) as above
# 6 inserts, 1 delete, 1 update to observe how parquet files creating & how log folder is getting updating
# In deletion and updation, older versioned files and deleted files never be considered in futhermore operations, they will
retain for some period due to soft deletion policy.
# But log folder properly update all info which to be consider and which to be ignored. To smother meta data maintenance by
avoiding small small many number of parquet files, optimize technique often used to combine them into single optimize sized parquet
file in delta tables
%sql optimize scd2demo      # with this one keyword "optimize" it will combine all small parquet files which less than 1GB size into
single file
```

11. Vacuum

```
# Vacuum [ to clean up the too many small sized obsolete/soft deletion parquet files]
%sql vacuum scd2demo dry run      # After optimized, it will plan/list out all small unnecessary parquet files to be
clean up and kept remain combined optimized files only
%sql vacuum scd2demo retain 720 hours dry run      # [ Last 30 days files]
%sql set spark.databricks.delta.retentionDurationCheck.enabled = False      # if we wish to disable the restriction on
deletion of retention files
%sql vacuum scd2demo      # it really cleans up all small retentioned files
```

12. Z-Ordering

```
# Z-Ordering
# In this technique, before performing above optimization, First ordering, next optimization as combined parquet files, then scan of
needed files while querying becomes more easy to identify.
%sql optimize scd2demo zorder pk1      # pk1 is 1st column(primarykey-1 column)
```

13. Schema Evolution

```
# Schema Evolution: Merge Schema
# It means adding extra columns to delta table in future updates/inserts
# first create delta table(emp_demo) with data insertion, next create dataframe with some new data, new schema(Extra columns), then
normal write command will throw error, so use as below:
df.write.option("mergeSchema","true").format("delta").mode("append").saveAsTable("emp_demo")
%sql select * from emp_demo
```

14. Dataframe insert into Delta Table

```
# dataframe insert into delta table
df = spark.createDataFrame(data=data, schema=schema) --> display(df)
df.write.saveAsTable("default.fact_revenue")      # its a default managed delta table, not a specific path based
display(spark.table("default.fact_revenue"))      # Approach/Method-1 (to view delta table)
display(spark.sql("select * from default.fact_revenue"))      # Approach/Method-2 (to view delta table)
%sql select * from default.fact_revenue      # Approach/Method-3 (to view delta table)
# Inserting new data into the above existing delta table
data = [("RST Pvt Ltd","Q4",7000)]
schema = ["Company","Quarter","Revenue"]
df1 = spark.createDataFrame(data=data, schema=schema) --> display(df1)
df1.write.insertInto("default.fact_revenue", overwrite=False)      # means new records will be append into existing delta table
df1.write.insertInto("default.fact_revenue", overwrite=True)      # means all existing records get deleted/truncated, new
records only will be inserted into existing delta table
# SQL Method
data = [("RST Pvt Ltd","Q4",7000)]
schema = ["Company","Quarter","Revenue"]
df2 = spark.createDataFrame(data=data, schema=schema) --> display(df2)
df2.createOrReplaceTempView("nn_view1")      # Temp view created from df
%sql insert into default.fact_revenue select * from nn_view1      # inserted above temp view data into existing delta table
```

15. Identify Input files corrupt record

```
# Identify Input files of corrupt record
create ADLS with Folder(Input_File),sub-folders and csv files
Create Mount to access ADLS from databricks notebook
root_path = "/mnt/adls/Input_File/"
%fs ls /mnt/adls/Input_File/
from pyspark.sql import functions as F
df = spark.read.option("sep","|").option("header","true").csv(root_path + "/*/*/*") --. display(df)
dfNull = df.filter("ProductKey is Null") --> display(dfNull)
dfInputFileName = df.withColumn("input_file_name",F.input_file_name()).filter("ProductKey is Null")
display(dfInputFileName)
df1 = dfInputFileName.withColumn("date_ingested", F.current_timestamp()) --> display(df1)
```

16. UDF to check if Folder Exists

```
# UDF to check if Folder Exists
create ADLS with Folder(sales),sub-folders and csv files
Create Mount to access ADLS from databricks notebook
%fs ls /mnt/adls/sales
def folderExists(path):
    try:
```

```

        dbutils.fs.ls(path)
        return True
    except Exception as err:
        if 'java.io.FileNotFoundException' in str(err):
            return False
path = "mnt/adls/sales/2018/"
if not folderExists(path):
    print("Folder is not present, now its being created")
    dbutils.fs.mkdirs(path)
else:
    print("Folder is already present")

```

17. Window Functions (lead/lag)

```

# window Functions(Lead/Lag)
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, lead, lag
windowSpec = Window.partitionBy("department").orderBy("salary")
df.withColumn("lag",lag("salary",1).over(windowSpec)).show()
df.withColumn("lead",lead("salary",1).over(windowSpec)).show()

```

18.

```

# performance optimization Join Types: 1. Broadcast Join      2. Sort Merge Join      3. Shuffle Hash Join
# Sort Merge Join (SMJ): shuffle --> sort --> Merge
dfJoin.explain()      # it is to explain how (shuffle+sort+join) plan executes in spark
# Bucketing (for performance optimization)
# Instead of multiple shuffle, sorts, to improve performance this bucketing is used.
# partition is done on low cardinality columns i.e. less unique values(many duplicates needed, country, dept...etc, not primary keys)
# when Table having maximum high cardinality columns(most unique valued columns), bucketing is preferable
spark.conf.get("spark.sql.sources.bucketing.enabled")
from pyspark.sql.functions import col, rand
df = spark.range(1,10000,1,10).select(col("id").alias("PK"), rand(10).alias("Attribute")) --> display(df)      # here 1 to 10,000 numbers
# creating with increment of 1 and in 10 partitions
df.count()
df.write.format("parquet").saveAsTable("nonbucketedTable")      # here 10 Folders created

```

19. Creating Bucketed Table

```

# Creating Bucketed table
df.write.format("parquet").bucketBy(10,"PK").saveAsTable("bucketedTable")      # here 10 buckets creating for each partition, means total
10*10=100 folders
df1 = spark.table("bucketedTable")
df2 = spark.table("bucketedTable")
df3 = spark.table("nonbucketedTable")
df4 = spark.table("nonbucketedTable")
df3.join(df4, "PK", "inner").explain()      # by default Broadcast Join, if Less than 10Mb
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)      # it disables Broadcast Join
spark.conf.set("spark.sql.adaptive.enabled", False)
display(df3.join(df4, "PK", "inner"))
df3.join(df1, "PK").display()
df3.join(df1, "PK").explain()
df1.join(df2, "PK").display()
df1.join(df2, "PK").explain()

```

20. Convert df columns into dictionary

```

# Convert df columns into dictionary(Map Type)
df = spark.createDataFrame(data=data, schema=schema) --> display(df)
from pyspark.sql.functions import *
dfDict =
df.select(col("ProductID"),col("ProductName"),col("unitPrice"),col("DiscountPercent"),create_map(col("ProductName"),col("UnitPrice"))).a
lias("PriceDict"))
display(dfDict)
dfDict = df.withColumn("PriceDict",create_map(lit("ProductName"),col("ProductName"),lit("UnitPrice"),col("UnitPrice")))) -->
display(dfDict)

```

21. Delta Cache

```

# Delta Cache (Performance optimizer)
# spark Cache is in-memory persist, But Delta Cache is disk persist and more efficient than spark cache
schema = StructType([StructField("Year", IntegerType(),StructField("FirstName"...)])
df =
spark.read.option("sep",",").option("header",True).schema(schema).csv("/FileStore/tables/bigdata_training/read_files/Baby_Names.csv") -
-> display(df)
df.write.format("delta").saveAsTable("names")
spark.conf.set("spark.databricks.io.cache.enabled","true")
spark.conf.get("spark.databricks.io.cache.enabled")
%sql select * from names      # here first time data accessing from remote [ DBFS/ADLS/S3/GCS...etc] and caching entire delta table
into local databricks disk
%sql select * from names where Year = 2007      # here data accessing from cache, not from remote

```

22. Convert df Array Elements into columns

```

# Convert df Array Elements into Columns
df = spark.createDataFrame(sc.parallelize([[ 'ABC',[1,2,3]], [ 'XYZ',[2,None,4]], [ 'KLM',[8,7]], [ 'IJK',[5]]], [ "key","value"])) -->
display(df)
df.select("key",df.value[0],df.value[1],df.value[2]).display()      # if array elements Less, this OK, but 100's of elements we need
below method
from pyspark.sql.functions import *

```

```
dfSize = df.select("key","value",size("value").alias("NoOfArrayElements")) --> display(dfSize)
max_value = dfSize.agg({"NoOfArrayElements":"max"}).collect()[0][0] --> print(max_value)
def arraySplitIntoCols(df,maxElements):
    for i in range(maxElements):
        df = df.withColumn(f"new_col_{i}",df.value[i])
    return df
dfout = arraySplitIntoCols(df,max_value) --> display(dfout)
```

23. Write df output into specific File Name

```
# Write df output into specific File Name
create Mount to access files of ADLS
%fs ls /mnt/adls/sales
salesDF = spark.read.format("parquet").load("/mnt/adls/sales/salesdata.parquet") --> display(salesDF)
print(salesDF.count())
from pyspark.sql.window import Window
from pyspark.sql.functions import *
windowSpec = Window.partitionBy("SalesOrderLineKey").orderBy("SalesOrderLineKey")
outDF = salesDF.withColumn("row_number",row_number().over(windowSpec)) --> display(outDF)
outDF.write.format("csv").option("header",True).option("sep","|").save("/mnt/adls/sales/outputOriginal/")
outDF.coalesce(1).write.format("csv").option("header",True).option("sep","|").save("/mnt/adls/sales/outputSingleFile/")
%fs rm -r /mnt/adls/sales/outputSingleFile/ # to clean up the output folder
# outDF.coalesce(1).write.format("csv").option("header",True).option("sep","|").save("/mnt/adls/sales/outputSingleFile/nn_result.csv")
# Note: this command gives specific Named Folders only, not specific named files, so we need to use pandas functions here as below
pandasDF = outDF.toPandas() --> print(pandasDF)
outname = 'salesOutput.csv'
outdir = '/dbfs/mnt/adls/sales/'
pandasDF.to_csv('/dbfs/mnt/adls/sales/OutputSingleSpecificName/salesOutput.csv',index=False) # to run this, OutputSingleSpecificName
folder should present in adls/sales path
%fs head /mnt/adls/sales/OutputSingleSpecificName/salesOutput.csv
```

24. Workspace objects access control

```
# databricks Workspace objects access control
Add User in Azure AD --> Create Users/Groups in databricks --> Grant access(Manage/Edit/Run/Read)
databricks --> workspace --> shared (in this, if we kept our Notebooks/files, all users can access)
databricks --> workspace --> Users --> different emails(users), each having their own files/Notebooks..etc
click on specific user email(abc@gmail.com) --> permissions --> admins, abc@gmail.com are having Manage access, below that, we can give
various(Read/Run/Edit/Manage) accesses to other users/Groups
```

25. Workflows

```
# Workflows (Job scheduling)
# workflow: Running of multiple tasks
Create Notebook(Azure SQL DB --> ADLS) jdbc config --> Read as DF from SQL DB --> Create Mount for ADLS --> Write DF into ADLS as csv
from datetime import datetime
outPath = '/mnt/adls/empOutput_'+datetime.now().strftime("%Y%m%d%H%M%S")+ '/'
empDF.write.format("csv").option("header",True).option("sep","|").save(outPath)
# Note: here write command only mentioned, remain jdbc config, sql db read, mount all are as previous videos
Workflows --> Jobs --> create Job[Task Name, Type(Notebook), Source(workspace/github), Path(select the path),Cluster(Job-Cluster/All-
Purpose), Parameters(optional), Adv-options ( dependancy-libraries / Notifications / Edit retry policy)] --> create
Next click on Big + symbol for adding more tasks like above task --> Finally click Run Now or Schedule
Task1{SQLDB --> ADLS Bronze} + Task2{ADLS Bronze --> ADLS Silver} + Task3{ADLS Silver --> ADLS/Synapse Gold}...etc # here for each
task separate notebooks need to be create & add, finally all to be run as single Workflow Job
Its similar to GCP dataflow Spark Jobs submission ( flow chart will displayed)
```

26. ETL Mini Project (SQLDB --> ADLS) with ADF, keyvault

```
# ETL mini project [SQLDB --> ADLS] by Notebook with ADF trigger with KeyVault integration
in Azure SQL DB, we need to create 2 tables dbo.emp, dbo.pipe_log
dbutils.widgets.text("outputPath","")
tableName = dbutils.widgets.get("table")
outputLocation = dbutils.widgets.get("outputPath")
print(tableName)
Next jdbc config code with keyvault secret scopes,
empDF = spark.read.format("jdbc").option("url",jdbcUrl).option("dbtable",tableName).load() --> display(empDF) # SQL-DB Read df
Next Mount ADLS code
empDF.write.format("csv").option("header",True).option("sep","|").save(outputLocation)
total_rows = empDFcount() --> print(total_rows)
dbutils.notebook.exit(total_rows) # returning output parameter(ADLS output path) in ADF to use for next ADF activities (by
storing in SetVariable)
SetVariable1[@concat('/mnt/adls/empOutput_',formatDateTime(utcnow(),'yyyyMmddHHmmSS'),)]
SetVariable2[sqlTableName,value(dbo.emp)]
Notebook-Activity[Notebook link service, notebook path, Base parameters(table,@variables('sqlTableName'), outputPath,
@variables('Output_location'))]
SetVariable3[Record_Count, @string(activity('Adb_Activity').output.run.output)] # Adb_Activity is above Notebook Activity name in
ADF, we can give any name
Copy[source(ADLS, output Folder), sink(SQL DB, dbo.pipe_log table), check columns Mapping]
in above copy activity, under source additional columns Pipeline_Name(@pipeline().Pipeline), Table_Name(@variables('sqlTableName')),
Output_Location(@variables('Output_location')), Processed_Time(@utcnow()), Record_Count(@variables('Record_Count'))
```

27. Run Notebook in ADF by Event Trigger

```
# Notebook run in ADF with Event based Trigger
root_path = "/mnt/adls/event_trigger"
lst = dbutils.fs.ls(root_path)
latest_file = sorted(lst, reverse=True)[0] --> print(latest_file) # here ADLS files, SQL DB table schema both should be same columns
root_path += latest_file.name --> print(root_path) # Getting Most recent uploaded file in ADLS
df = spark.read.option("sep","|").option("header",True).csv(root_path) --> display(df) # Most Recent file reading df
```

```
jdbc config code
df.write.format("jdbc").option("url",jdbcUrl).option("dbtable","dbo.emp").option("user",jdbcusername).option("password",jdbcpassword).mode("append").save()
Next in Subscription --> Resource Providers --> Microsoft.EventGrid should be register, Next in ADF, config Notebook activity and Event based(Storage Events) trigger
```

28. Spark-SQL

```
# Query df using Spark SQL
productDF = spark.createDataFrame(data,schema=schema) --> display(productDF)    # here data,schema need to be defined
# old approach
productDF.createOrReplaceTempView("v_product")
%sql select * from v_product
# New Approach
sqlDF = spark.sql("select * from {table}", table = productDF) --> display(sqlDF)
sqlDF = spark.sql("select {column} from {table}", table=productDF, column=productDF["ProductName"]) --> display(sqlDF)
# transform with spark SQL
transformDF = spark.sql("select ProductID, concat(ProductName,Category), Price+StockQunatity as Total_Cost from {table}",
table=productDF) --> display(transformDF)
# transform with PySpark
from pyspark.sql.functions import col,concat,expr
transformDF =
productDF.select(col("ProductID"),concat(col("ProductName"),col("Category")).alias("ProductNameCategory"),(expr("Price*StockQuantity")).alias("Total_Cost")) --> display(transformDF)
# Join df here
from pyspark.sql import Row
from pyspark.sql.functions import expr
product_df = spark.createDataFrame(product_data)    # here product_data,schema need to be defined
sales_df = spark.createDataFrame(sales_data)    # here sales_data,schema need to be defined
joinDF = spark.sql("select * from {table1} a join {table2} b on a.{joiningkey} = b.{joiningkey}", table1=product_df, table2=sales_df,
joiningkey=product_df["product_id"])
display(joinDF)
```

29. Except Columns

```
# Except Columns in saprk SQL
from pyspark.sql.function import rand
df = spark.createDataFrame(data,schema) --> display(df)
df.createOrReplaceTempView("Products")
%sql select * from Products
%sql select * except(category, active) from Products
%sql select p.* except(brand), c.* except(customer_id) from product p join customer c on c.purchased_product_id = p.product_id
# same above in PySpark
df.select([col for col in df.columns if col not in {'category','active'}]).display()
```

30. Auto-Loader

```
# Auto-Loader (Incremental data load in databricks)
# Water Mark Method: It continuously check for new files even arrived/not.
# Check Point Method: It creates checkpoints using offset values, but its not best suited for large datasets, then we need to go for alternatives (Logic Apps, Event Grid...etc)
# Auto-Loader: Its best solution provided by databricks. continuously, efficiently, automatically loading new files as they arrive
# It stores the Metadata of processed files through its internal Database(RocksDB)
# Processing new files immediately using spark structured streaming
# using advanced optimized Cloud-Native components to identify new files as soon as they arrived
# Spark: 1 API call per 1 folder Ex: if we consider hourly one file, yearly 87610 new files need 8761 API calls hit
# Auto-Loader: 5000 files at a file from ADLS, 18 API hits enough for 87610 files in a year
SubscriptionID --> RG --> TenantID --> Service Principal ( ClientID & Client Secret )    # Azure Queue creation
# AutoLoader architecture Method-1: ADLS --> Event Grid --> Azure Queue --> RocksDB --> Spark Streaming --> Delta Lake
# AutoLoader architecture Method-2: ADLS --> Directory Listing --> RocksDB --> Spark Streaming --> Delta Lake
adlsAccount = dbutils.secrets.get(scope="keyVault secret", key="adls name")
adlsKey = dbutils.secrets.get(scope="keyVault secret", key="adls access key")
spark.conf.set("fs.azure.account.key." + adlsAccount + ".blob.core.windows.net", adlsKey)
spark.conf.set("fs.azure.account.key." + adlsAccount + ".dfs.core.windows.net", adlsKey)
df = spark.readStream.format("cloudFiles").option("cloudFiles.format","json").load(location)
df = spark.readStream.format("cloudFiles").option("cloudFiles.format","json").schema(user_schema).load(location)
df = spark.readStream.format("cloudFiles").option("cloudFiles.format","json").option("cloudFiles.schemaLocation",
'/mnt/data/inferred_schema').option("cloudFiles.inferColumnTypes","true").load(location)
```

31. Pivot/Unpivot

```
# Pivot/Unpivot
pivot_df = df.groupBy("Company").pivot("Quarter").sum("Revenue") --> display(pivot_df)
unpivot_df = pivot_df.selectExpr("Company", "stack(4,'Q1',Q1,'Q2',Q2,'Q3',Q3,'Q4',Q4) as (Quarter, Revenue)") --> display(unpivot_df)
```

32. Union/Unionall

```
# Union/UnionAll
union_df = df1.union(df2) --> display(union_df) --> display(union_df.dropDuplicates())
unionAll_df = df1.unionAll(df2) --> display(unionAll_df)
Note: if df1 having 6 columns, df2 having 5 columns, it will failed due to schema mismatch
```

33. Joins

```
# Joins
join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"inner") --> display(join_df)
join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"full") --> display(join_df)    #
full/fullouter/outer/full_outer
join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"left") --> display(join_df)    # Left/Leftouter,Left_outer
```

```

join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"right") --> display(join_df)      #
right/rightouter/right_outer
join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"semi") --> display(join_df)      #
leftsemi/left_semi/semi    only common records of left table
join_df = employeeDF.join(deptDF,employeeDF.emp_dept_id == deptDF.dept_id,"anti") --> display(join_df)      #
leftanti/left_anti/anti    only uncommon records of left table

```

34. Explode

```

# Explode
df2 = df_app.select(df_app.name, explode(df_app.Appliances)) --> display(df2)      # exploding array here schema (string, array) --->
(string, string), here Null values ignored
df3 = df_brand.select(df_brand.name, explode(df_brand.Brand)) --> display(df3)      # exploding array here schema (string,
map(key:value)) ----> (string, string(key), string(value)) here Null values ignored
display(df_app.select(df_app.name, explode_outer(df_app.Appliances)))      # same as explode but here Null values also comes
in output
display(df_brand.select(df_brand.name, explode_outer(df_brand.Brand)))      # same as explode but here Null values also comes
in output
display(df_app.select(df_app.name, posexplode(df_app.Appliances)))      # same as explode but here position column extra
will be comes in output, null values ignored
display(df_brand.select(df_brand.name, posexplode(df_brand.Brand)))      # same as explode but here position column extra
will be comes in output, Null values ignored
display(df_app.select(df_app.name, posexplode(df_app.Appliances)))      # same as posexplode, but here Null values also
comes in output
display(df_brand.select(df_brand.name, posexplode(df_brand.Brand)))      # same as posexplode, but here Null values also
comes in output

```

35. Read Flatten Json Files

```

# Reading & Flatten Json files
df = spark.read.option("multiline", "true").json("/FileStore/tables/json_files/Sample_Data.json") --> display(df)
def flatten(df):
    compute Complex Fields (Lists and Structs) in Schema
    complex_fields = dict([(field.name, field.dataType)
                           for field in df.schema.fields
                           if type(field.dataType) == ArrayType or type(field.dataType) == StructType])

    while len(complex_fields)!=0:
        col_name=list(complex_fields.keys())[0]
        print ("Processing :"+col_name+" Type : "+str(type(complex_fields[col_name])))

        if StructType then convert all sub element to columns.
        i.e. flatten structs
        if (type(complex_fields[col_name]) == StructType):
            expanded = [col(col_name+'_'+k).alias(col_name+'_'+k) for k in [ n.name for n in complex_fields[col_name]]]
            df=df.select("?", *expanded).drop(col_name)

        if ArrayType then add the Array Elements as Rows using the explode function
        i.e. explode Arrays
        elif (type(complex_fields[col_name]) == ArrayType):
            df=df.withColumn(col_name,explode_outer(col_name))

        recompute remaining Complex Fields in Schema
        complex_fields = dict([(field.name, field.dataType)
                               for field in df.schema.fields
                               if type(field.dataType) == ArrayType or type(field.dataType) == StructType])

    return df
df_flatten = flatten(df) --> display(df_flatten)

```

36. Bad Records Handling

```

# Bad Records handling
df = spark.read.format("csv").option("header", "true").option("inferSchema",
"true").load("/FileStore/tables/production_data_corrupt.csv")
display(df)
next define custom schema by using structType
# permissive mode
df = spark.read.format("csv").option("mode","permissive").option("header",
"true").schema(schema).load("/FileStore/tables/production_data_corrupt.csv")
display(df)      # in this case, it will highlight corrupted rows/records
# dropmalformed
df = spark.read.format("csv").option("mode","dropmalformed").option("header",
"true").schema(schema).load("/FileStore/tables/production_data_corrupt.csv")
display(df)      # in this case, it will ignore corrupted rows/records
# FailFast
df = spark.read.format("csv").option("mode","failfast").option("header",
"true").schema(schema).load("/FileStore/tables/production_data_corrupt.csv")
display(df)      # in this, it won't run and throw error

```

37. Access ADLS

```

# with ADLS [ service principal / AAD credential / ADLS access keys / Mount point]
create ADLS, take access key(xxxxxxxxxx)
# Method-1
spark.conf.set("fs.azure.account.key.nnadls2025.dfs.core.windows.net", "xxxxxxxxxxxx")
dbutils.fs.ls("abfss://bronze@nnadls2025.dfs.core.windows.net/")
file_location = "abfss://bronze@nnadls2025.dfs.core.windows.net/file1.csv"
df = spark.read.format("csv").option("inferSchema","true").option("header","true").option("delimiter",",").load(file_location)

```



```
display(df)
# Method-2 (Mount ADLS)
dbutils.fs.mount.(
    source = "wasbs://bronze@nnadls2025.blob.core.windows.net",
    mount_point = "/mnt/nn_adls",
    extra_configs = {"fs.azure.account.key.nnadls2025.blob.core.windows.net":"xxxxxxx"}
)
dbutils.fs.ls("/mnt/nn_adls")
dbutils.fs.mounts()
dbutils.fs.unmount("/mnt/nn_adls")
```

38. Read Tables from Azure SQL DB

```
# Read Tables from Azure SQL DB
Create Azure SQL DB with sample table/create new table with data and allow firewall IP
# Method-1
jdbcHostname = "xxxxxxxxxxxxxx" # SQL DB server name
jdbcPort = 1433
jdbcDatabase = "nn-db"
jdbcUsername = "naren"
jdbcPassword = "naren123"
jdbcDriver = "com.microsoft.sqlserver.jdbc.SQLServerDriver"
jdbcUrl = f"jdbc:sqlserver://{jdbcHostname}:{jdbcPort};databaseName={jdbcDatabase};user={jdbcUsername};password={jdbcPassword}"
df1 = spark.read.format("jdbc").option("url", jdbcUrl).option("dbtable", "SalesLT.Product").load()
display(df1)
# Method-2
Azure SQL DB --> connection string --> copy jdbc connection string
connectionString = "xxxxxxxxxxxxxx"
df = spark.read.jdbc(connectionString, "SalesLT.Product") --> display(df)
```

39. ETL pipeline Azure SQL DB --> ADLS

```
# ETL pipeline from Azure SQL DB --> ADLS [ ADLS Mount included here]
Extract: Read Tables from Azure SQL DB (jdbc connector/connectionString), create Dataframes(df)
Transformation: Null replaced by default value, remove Duplicates, perform Join & Aggregations
Load: Create Mount Point for ADLS, Data loading into ADLS as parquet files
df_product = spark.read.format("jdbc").option("url", jdbcUrl).option("dbtable", "SalesLT.Product").load()
display(df_product)
df_sales = spark.read.format("jdbc").option("url", jdbcUrl).option("dbtable", "SalesLT.SalesOrderDetail").load()
display(df_sales)
df_product_cleansed = df_product.na.fill ( {"Size":"M", "Weight":100} ) --> display(df_product_cleansed) # Dimension Dataframe
df_sales_cleansed = df_sales.dropDuplicates() --> display(df_sales_cleansed) # Fact Dataframe
df_join = df_sales_cleansed.join(df_product_cleansed, df_sales_cleansed.ProductID == df_product_cleansed.ProductID, "leftouter")
                        .select(df_sales_cleansed.ProductID, df_sales_cleansed.UnitPrice,
                                df_sales_cleansed.LineTotal, df_product_cleansed.Name,
                                df_product_cleansed.Color, df_product_cleansed.Size, df_product_cleansed.Weight)

display(df_join)
df_agg = df_join.groupBy(["ProductID", "Name", "Color", "Size", "Weight"]).sum("LineTotal").withColumnRenamed("sum(LineTotal)",
"sum_total_sales")
display(df_agg)
dbutils.fs.mount.(
    source = "wasbs://bronze@nnadls2025.blob.core.windows.net",
    mount_point = "/mnt/nn_adls",
    extra_configs = {"fs.azure.account.key.nnadls2025.blob.core.windows.net":"xxxxxxx"}
)
dbutils.fs.ls("/mnt/nn_adls")
# write the data in parquet format
df_agg.write.format("parquet").save("/mnt/nn_adls/nn_parquet/") # this file will be created in ADLS in nn_parquet folder inside
nnadls2025 container
# write the data in csv format
df_agg.write.format("csv").option("header", "true").save("/mnt/nn_adls/nn_csv/") # this file will be created in ADLS in nn_csv
folder inside nnadls2025 container
```

40. Key Vault Integration with databricks

```
# Azure key vault Integration with Databricks
Create Azure Key Vault and secret inside it. kept copy keyvault secret name & its value # secret name: nn_password, value:
Narian@318
Create Scoped credential, for this use url: https://community.cloud.databricks.com/?o=3418180493285762#secrets/createScope # here
3418**** is our workspace id, my scope name is nn_scope1
Azure KeyVault --> properties { Vault URI(DNS name), Resource ID }, these will be used while secret scope creation in above url #
its not worked in community edition
dbutils.secrets.get(scope = "nn_scope1", key = "nn_password")
Ex: jdbcPassword = dbutils.secrets.get(scope = "nn_scope1", key = "nn_password")
    jdbcDatabase = dbutils.secrets.get(scope = "nn_scope1", key = "nn_dbname") # here nn_dbname value is nn_db
    jdbcUsername = dbutils.secrets.get(scope = "nn_scope1", key = "nn_user") # here nn_user value is naren
```

41. Streaming data

```
# Streaming data
define schema by using structType...etc
dbutils.fs.mkdirs("/Filestore/tables/stream_checkpoint/")
dbutils.fs.mkdirs("/Filestore/tables/stream_read/")
dbutils.fs.mkdirs("/Filestore/tables/stream_write/")
df = spark.readStream.format("csv").schema(schema).option("header", "true").option("sep", ";").load("/FileStore/tables/stream_read/")
```

```
df1 = df.groupBy("Shop").sum("Sale_count") --> display(df1) # after upload our input files one by one, and observe output by run this command
df4 = df.writeStream.format("parquet").outputMode("append").option("path",
"/FileStore/tables/stream_write/").option("checkpointLocation", "/FileStore/tables/stream_checkpoint/").start().awaitTermination()
display(spark.read.format("parquet").load("/FileStore/tables/stream_write/*.parquet"))
```

42. Repartition, Coalesce

```
# Re-Partition, Coalesce
sc.defaultParallelism
spark.conf.get("spark.sql.files.maxPartitionBytes")
from pyspark.sql.types import IntegerType
df = spark.createDataFrame(range(10), IntegerType())
df.rdd.getNumPartitions() # output: 8
df.rdd.glom().collect()
df1 = df.repartition(20) --> df.rdd.getNumPartitions() # output: 20 # repartition used to increase/decrease partition numbers
df1 = df.repartition(2) --> df.rdd.getNumPartitions() # output: 2
df2 = df.coalesce(2) --> df.rdd.getNumPartitions() # output: 2 # coalesce used to reduce only
dbutils.fs.ls("/FileStore/tables/baby_names/")
df = spark.read.format("csv").option("inferSchema", True).option("header", True).option("sep", "," ).load("/FileStore/tables/baby_names/")
df.rdd.getNumPartitions() # output: 3 [ bcoz each file size having below 500 kb, not more than 128Mb (default partition size)]
spark.conf.set("spark.sql.files.maxPartitionBytes", 100000) # here we config default size as 100 kb in place of 128 Mb
spark.conf.get("spark.sql.files.maxPartitionBytes")
df = spark.read.format("csv").option("inferSchema", True).option("header", True).option("sep", "," ).load("/FileStore/tables/baby_names/")
df.rdd.getNumPartitions() # output: 14 [ 1st file: 484 kb(5 partitions), 2nd file: 468 kb(5 partitions), 3rd file: 362 kb(4 partitions)]
```

43. Cache, Persist

```
# Cache, Persist
Cache: it gives option to store data in-memory across nodes
Persist: it gives option to store data either in-memory/in disc across nodes
# without Cache/Persist: [df1-->df2-->df3], [df1-->df2-->df4], [df1-->df2-->df5]
# with df2 Cached/Persisted: [df2-->df3], [df2-->df4], [df2-->df5]
Broadcast-Variable: In this, full copy of data sent to each worker node instead of partitions. Its suitable for small sized tables only
Transaction_df = spark.createDataFrame(data = Transaction, schema = ['Store_id','Item','Amount']) --> Transaction_df.show()
Store_df = spark.createDataFrame(data = Store, schema = ['Store_id','Store_name']) --> Store_df.show()
from pyspark.sql.functions import broadcast
join_df = Transaction_df.join(broadcast(Store_df),Transaction_df['Store_id'] == Store_df['Store_id']) --> join_df.show()
join_df.explain(true) # it displays Parsed/unresolved Logical Plan, Analyzed Logical Plan, Optimized Logical Plan, Physical Plan
```

44. AQE (Adaptive Query Execution)

```
# Adaptive Query Execution(AQE) performance tuning
spark.conf.set("spark.sql.adaptive.enabled", True)
```

45. Handling Null Values

```
# Handling Null Values
display(df.filter(df.Mark.isNull())) (or) display(df.filter("Mark IS NULL")) (or) display(df.filter(col("Mark").isNull()))
display(df.filter(df.Mark.isNotNull()))
display(df.filter(df.Mark.isNotNull()) & (df.Attendance.isNotNull()))
display(df.filter(df.Mark.isNotNull()) | (df.Attendance.isNotNull()))
display(df.na.drop()) (or) display(df.dropna("any"))
display(df.na.drop("all"))
display(df.na.drop(subset =["Mark", "Attendance"]))
display(df.na.fill(value=0)) (or) display(df.fillna(value=0)) # it replaces all Null values with zero for all Integer type columns only, Not for string columns
display(df.na.fill(value="NA")) # it replaces all Null values with zero for all string columns only, Not for integer columns
display(df.na.fill(value=0, subset =["Mark", "Attendance"]))
display(df.na.fill( { "Mark": 0, "Status": "NA", "Name": "No_Name", "Subject": "English", "Attendance": 50} ) )
```

46. UDF

```
# UDF (Renameing all columns)
emp_df = spark.createDataFrame(data = employee_data, schema = emp_schema)
import pyspark.sql.functions as f
def rename_columns(rename_df):
    for column in rename_df.columns:
        new_column = "col_" + column
        rename_df = rename_df.withColumnRenamed(column,new_column)
    return rename_df
renamed_df = rename_columns(emp_df) --> display(renamed_df)
# (Lower --> upper case)
import pyspark.sql.functions import upper, col
def uppercase_col(df):
    emp_df_upper = df.withColumn('name_upper',upper(df.Name))
    return emp_df_upper
up_case_df = uppercase_col(emp_df) --> display(up_case_df)
```

47. Data Skew

```
# Data skew
(Germany: 10M records),(France: 0.5M records),(Italy: 0.1M records),(Sweden: 50k records) schema = (country, data count per day)
when un-even distribution happens, some taks finished quickly, some tasks may be delayed with huge records, so it looses advantages of parallelism due to data skew
To handle Data skew problems: 1. Salt the skewed column 2. Apply Skew-hint 3. Use Broadcast Join 4. Enable Adaptive Query Execution(AQE)
Salting: subdividing the huge partition into small sub-partitions by using Pseudo Id # in this case, Germany-1, Germany-2, Germany-3, France, Italy, Sweden
Skew-Hint:
```

```
select /*+ SKEW('orders', 'o_custId',0) */ *                # single column, single skew value
  from orders, customers where o_custId = c_custId
select /*+ SKEW('orders', 'o_custId',(0,1,2)) */ *          # single column, multiple skew values
  from orders, customers where o_custId = c_custId
select /*+ SKEW('orders', ('o_custId', 'o_storeRegionId'), ((0,1001),(1,1002))) */ *          # multiple columns, multiple skew values
  from orders, customers where o_custId = c_custId and o_storeRegionId = c_regionId
```

AQE: It will automatically split the huge partition into even sized small partitions without manual intervention

48. Shuffle Parameter

```
# Shuffle Parameter
spark.conf.get("spark.sql.shuffle.partitions")    # default: 200 partitions
spark.conf.get("spark.sql.shuffle.partitions",20) # custom value: 64 partitions    for this, its better to follow multiples of cores, if we have 16 cores, we can split 16/32/48/64...etc partitions
```

49. Optimize Auto Scaling

```
# Optimize Auto Scaling
Databricks will choose no of worker nodes automatically as per load within range of 2 to 8 Nodes at starting, next slowly scale up to max 64
AutoScaling: Optimized, Standard
JobCompute-Clusters: always use optimized autoscaling
AllPurpose-Clusters: use standard autoscaling
```

50. Check point

```
# Dataframe Checkpoint
It is performance optimization technique. to avoid recomputing for 100's of Transformations, we can create checkpoints at several points.
Cache is also similar to this, but it returns intermittent data into in-memory and we can retrieve lineage Graph at any point of time, but in checkpoint returns into disk and after checkpoint, we can't get previous lineage Graph. At any Failures, we can recover past data from these graphs by recomputing
```

51. Split

```
# Split
df1 = emp_df.withColumn('FirstName',split(emp_df['Name'],' ').getItem(0)).withColumn('LastName',split(emp_df['Name'],' ').getItem(1)).drop(emp_df['Name'])
display(df1)
split_col = pyspark.sql.functions.split(emp_df['doj'], '-')
df2 = emp_df.select("emp_id","Name","emp_dept_id","salary", split_col.getItem(0).alias('joining_year'),
split_col.getItem(1).alias('joining_month'), split_col.getItem(2).alias('joining_day'))
display(df2)
```

52. Array_zip

```
# Arrays_zip [part-1]
schema =(Name,Array_Score1,Array_Score2), data=(Mike,[4,6,7,9,2],[1,2,3,7,7]) --> zipped_value [{4,1},{6,2},{7,3},{9,7},{2,7}]
output_df = input_df.withColumn("zipped_value", arrays_zip("Array_Score1", "Array_Score2"))
array_data = [("John",4,1),("John",6,2),("David",7,3),("Mike",3,4)]
array_schema =["Name", "Score1", "Score2"]
array_df = spark.createDataFrame(data=array_data, schema=array_schema)
master_df = array_df.groupBy("Name").agg(F.collect_list('Score1').alias('Array_Score1'), F.collect_list('Score2').alias('Array_Score2'))
display(master_df)
arr_zip_df = master_df.withColumn("zipped_value", F.arrays_zip("Array_Score1", "Array_Score2")) --> arr_zip_df.show(10,False)

df_brand = spark.createDataFrame(data=emp_df, schema =['Department','Employee'])
df_brandZip = df_brand.withColumn("zip",F.arrays_zip(df_brand['Employee'])) --> display(df_brandZip)
df_brand_exp = df_brandZip.withColumn("Explode",F.explode(df_brandZip.Zip)) --> display(df_brand_exp)
df_brand_output = df_brand_exp.withColumn("employee_emp_name",df_brand_exp['Explode.Employee.emp_name'])
                                .withColumn("employee_yrs_of_service",df_brand_exp['Explode.Employee.yrs_of_service'])
                                .withColumn("employee_salary",df_brand_exp['Explode.Employee.salary'])
                                .withColumn("employee_Age",df_brand_exp['Explode.Employee.Age']).drop("Explode").drop("zip").drop("Employee")
display(df_brand_output)
```

53. Join by Python function

```
# Join by Python Function
column_list = emp_df.columns --> print(column_list)
joined_string = ",".join(column_list) --> print(joined_string)
# Use Cases: To List out data before doing join, update, insert operations
joinstring = "AND".join(list(map(lambda x: ("Target." + x + " = Source."+x), column_list))) --> print(joinstring)
updatestring = ",".join(list(map(lambda x: ("Target." + x + " = Source."+x), column_list))) --> print(updatestring)
sourceinsertstring = ",".join(list(map(lambda x: ("Source."+x), column_list))) --> print(sourceinsertstring)
```

54. Partition by

```
# Partition by
df.write.partitionBy(key).csv(path)
df =
spark.read.format("csv").option("inferSchema",True).option("header",True).option("sep","," ).load("/FileStore/tables/baby_names_input/")
display(df)
df.groupBy("Year").count().show(truncate=False)
df.write.option("header",True).partitionBy("Year").mode("overwrite").csv("/FileStore/tables/baby_names_output")    # it gives year wise partitions
dbutils.fs.rm("/FileStore/tables/baby_names_output/", True)
dbutils.fs.mkdirs("/FileStore/tables/baby_names_output/")
df.write.option("header",True).partitionBy("Year","Sex").mode("overwrite").csv("/FileStore/tables/baby_names_output")    # it gives 16 partitions (year wise(8) and gender(2) wise = 8*2=16), 8 Folders with 2 sub-Folders each
```

```

dbutils.fs.rm("/FileStore/tables/baby_names_output/", True)
dbutils.fs.mkdirs("/FileStore/tables/baby_names_output/")
df.write.option("header",True).option("maxRecordsPerFile",4200).partitionBy("Year").mode("overwrite").csv("/FileStore/tables/baby_names_output")

df =
spark.read.format("csv").option("inferSchema",True).option("header",True).option("sep",",").load("/FileStore/tables/baby_names_test/BabyNames.csv")
display(df) --> print(df.count())
print(df.rdd.getNumPartitions())
from pyspark.sql.functions import spark_partition_id
df.withColumn("partitionId", spark_partition_id()).groupBy("partitionId").count().show()
df_5 = df.select(df.year,df.Country,df.Sex,df.Count).repartition(5) # it means PartitionIds from 0 to 4
df_5.withColumn("partitionId", spark_partition_id()).groupBy("partitionId").count().show()

```

55. Null Count of Each column

```

# Null Count of Each column
from pyspark.sql.functions import col,count,when
result_df = df.select([count(when(col(c).isNull(),c)).alias(c) for c in df.columns]) --> display(result_df)

```

56. Top/Bottom 'n' rows

```

# Top/Bottom 'n' Rows Group wise
from pyspark.sql.window import window
from pyspark.sql.functions import *
windowDept = Window.partitionBy("name").orderBy(col("Mark").desc())
df2 = df.withColumn("row",row_number().over(windowDept)).orderBy("name","row") --> display(df2)
df3 = df2.filter(col("row") <= 2) --> display(df3) # it gives Top 2 records Name Group wise
windowDept = Window.partitionBy("Subject").orderBy(col("Mark").desc())
df4 = df.withColumn("row",row_number().over(windowDept)).orderBy("name","row") --> display(df4)
df5 = df4.filter(col("row") <= 2) --> display(df5) # it gives Top 2 records Subject Group wise
Note: if we need Bottom 'n' records, remove desc() in above commands

```

57. Greatest,Least,Min,Max

```

# Greatest, Max, Least, Min
Greatest: Highest of all columns for a specific person Ex: In which subject has scored high marks, Least means lowest scored subject
Max: Highest of all rows for a specific subject among all students Ex: Who is Topper in Physics?, Min means Top Looser in Physics
from pyspark.sql.functions import *
great_df = df.withColumn("Greatest", greatest("Subject_1","Subject_2","Subject_3","Subject_4","Subject_5")) --> display(great_df)
least_df = df.withColumn("Least", least("Subject_1","Subject_2","Subject_3","Subject_4","Subject_5")) --> display(least_df)
max_df = df.agg({'Subject_1': 'max'},{'Subject_2': 'max'},{'Subject_3': 'max'},{'Subject_4': 'max'},{'Subject_5': 'max'}) --> display(max_df)
min_df = df.agg({'Subject_1': 'min'},{'Subject_2': 'min'},{'Subject_3': 'min'},{'Subject_4': 'min'},{'Subject_5': 'min'}) --> display(min_df)

```